

Reinforcement Learning

CS60077 — Comprehensive Study Notes

From Markov Processes to Value Function Approximation

Covering Weeks 1–6

*Mathematical Preliminaries · Markov Decision Processes · Dynamic Programming
Monte-Carlo Methods · Temporal-Difference Learning · TD(λ)
On-/Off-Policy Control (Sarsa, Q-Learning) · Value Function Approximation*

Compiled from course slides (CS60077, Autumn 2026–27) with reference to
D. Silver, *UCL Course on RL* and R. Sutton & A. Barto, *Reinforcement Learning: An Introduction*
(2nd ed.)

September 8, 2026

Contents

1	Mathematical Preliminaries	4
1.1	Probability Theory	4
1.1.1	Sample space, events, and probability measure	4
1.1.2	Conditional probability and independence	4
1.1.3	Random variables and distributions	4
1.1.4	Multiple random variables	5
1.2	Linear Algebra	5
1.2.1	Matrices and basic operations	5
1.2.2	Norms	5
1.2.3	Eigenvalues, positive semi-definiteness	5
1.2.4	Matrix calculus: gradients used later	6
1.2.5	Solving linear systems: why $(I - \gamma P)^{-1}$ appears	6
2	Introduction to Reinforcement Learning	7
2.1	What is Reinforcement Learning?	7
2.2	The Agent–Environment Interface	7
2.2.1	History and state	8
2.2.2	Full vs. partial observability	8
2.3	Inside an RL Agent	9
2.3.1	Taxonomy of RL agents	9
2.4	Learning and Planning, Prediction and Control	9
2.5	Exploration and Exploitation	10
3	Markov Processes and Markov Reward Processes	11
3.1	The Markov Property	11
3.2	Markov Chains	11
3.2.1	Multi-step transition probabilities	12
3.3	Markov Reward Processes	13
3.3.1	Return	13
3.3.2	Value function	13
3.3.3	The Bellman Equation for MRPs	14
3.3.4	Matrix form and the direct solution	15
4	Markov Decision Processes	16
4.1	Definition	16
4.2	Policies	16
4.3	Value Functions	17
4.3.1	The Bellman Expectation Equation	17
4.3.2	Matrix form	18
4.4	Optimal Value Functions	18
4.4.1	Existence and uniqueness of an optimal policy	18

4.4.2	The Bellman Optimality Equation	19
5	Planning by Dynamic Programming	21
5.1	Why Dynamic Programming Applies to MDPs	21
5.2	Iterative Policy Evaluation	21
5.2.1	Convergence: the contraction-mapping argument	22
5.3	Policy Iteration	23
5.3.1	The Policy Improvement Theorem	23
5.3.2	Modified / Generalised Policy Iteration	25
5.4	Value Iteration	25
5.4.1	Deterministic Value Iteration	25
5.4.2	Convergence of Value Iteration	26
5.5	Summary of Synchronous DP Algorithms	26
5.6	Asynchronous Dynamic Programming	26
6	Model-Free Prediction: Monte-Carlo Learning	28
6.1	Monte-Carlo Reinforcement Learning	28
6.1.1	First-Visit and Every-Visit MC	28
6.2	Incremental Mean and Incremental Monte-Carlo Updates	29
7	Temporal-Difference Learning and TD(λ)	31
7.1	TD(0): the Simplest Temporal-Difference Method	31
7.1.1	Advantages and disadvantages of MC vs. TD	32
7.1.2	Bias/Variance Trade-off	32
7.2	Batch MC and TD: the AB Example	33
7.3	Unifying View: Backup Diagrams	34
7.4	n -Step Prediction	34
7.4.1	Averaging n -step returns and the λ -return	35
7.5	Backward-View TD(λ): Eligibility Traces	35
7.5.1	Two special cases	36
7.5.2	Equivalence of Forward and Backward Views (Offline Updating)	36
8	Model-Free Control I: On-Policy Monte-Carlo Control	38
8.1	On- and Off-Policy Learning	38
8.2	Generalised Policy Iteration with Monte-Carlo Evaluation	38
8.2.1	Monte-Carlo Policy Iteration and Monte-Carlo Control	39
8.3	GLIE and Convergence to the Optimal Policy	40
9	Model-Free Control II: Sarsa and Sarsa(λ)	42
9.1	From MC Control to TD Control	42
9.2	Sarsa	42
9.3	Sarsa(λ)	43
10	Off-Policy Learning and Q-Learning	45
10.1	Why Off-Policy Learning?	45
10.2	Importance Sampling	45
10.2.1	Importance Sampling for Off-Policy Monte-Carlo	46
10.2.2	Importance Sampling for Off-Policy TD	46
10.3	Q-Learning	46
10.3.1	Off-Policy Control with Q-Learning	46
10.4	The Relationship Between DP and TD	48

11 Value Function Approximation	50
11.1 Value Function Approximation	50
11.2 Gradient Descent	50
11.2.1 Value function approximation by SGD	51
11.3 Linear Value Function Approximation	51
11.4 Incremental Prediction Algorithms	52
11.5 Control with Value Function Approximation	52
11.6 Convergence Issues: The Deadly Triad	53
11.7 Batch Methods	54
11.7.1 Least Squares Prediction and Experience Replay	54
11.7.2 Linear Least Squares Prediction: a closed-form solution	54
11.7.3 Least Squares Policy Iteration (LSPI)	55
A Summary Tables and Exam Cheat-Sheet	57
A.1 Notation Glossary	57
A.2 The Bellman Equation Family	57
A.3 Every Update Rule in One Table	57
A.4 Convergence Conditions, at a Glance	58
A.5 “If the Exam Asks You to Derive...”	58

Chapter 1

Mathematical Preliminaries

This chapter is a compressed reference for the probability theory and linear algebra used throughout the course. It is not meant to be read linearly like the rest of the notes — treat it as a lookup table for the tools that reappear in later derivations (expectations for return/value functions, matrices for the Bellman equation, gradients for value-function approximation).

1.1 Probability Theory

1.1.1 Sample space, events, and probability measure

Definition 1.1.1: Probability Space

A probability space is a triple $(\Omega, \mathcal{F}, \mathbb{P})$ where Ω is the *sample space* (set of all possible outcomes), $\mathcal{F}$ is the *event space* (a σ -algebra of subsets of Ω), and $\mathbb{P} : \mathcal{F} \rightarrow [0, 1]$ is a *probability measure* satisfying $\mathbb{P}(\Omega) = 1$ and countable additivity over disjoint events.

1.1.2 Conditional probability and independence

For events A, B with $\mathbb{P}(B) > 0$,

$$\mathbb{P}(A | B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)}.$$

Events A, B are *independent* iff $\mathbb{P}(A \cap B) = \mathbb{P}(A)\mathbb{P}(B)$, equivalently $\mathbb{P}(A | B) = \mathbb{P}(A)$.

Theorem 1.1.1: Bayes' Rule

$$\mathbb{P}(A | B) = \frac{\mathbb{P}(B | A) \mathbb{P}(A)}{\mathbb{P}(B)}.$$

Proof. Both $\mathbb{P}(A \cap B) = \mathbb{P}(A | B)\mathbb{P}(B)$ and $\mathbb{P}(A \cap B) = \mathbb{P}(B | A)\mathbb{P}(A)$ hold by the definition of conditional probability; equating the right-hand sides and dividing by $\mathbb{P}(B)$ gives the result. ■

1.1.3 Random variables and distributions

A random variable X is a measurable function $X : \Omega \rightarrow \mathbb{R}$. Its *cumulative distribution function* (CDF) is $F_X(x) = \mathbb{P}(X \leq x)$. For a discrete X the *probability mass function* (PMF) is $p_X(x) = \mathbb{P}(X = x)$; for a continuous X the *probability density function* (PDF) f_X satisfies $F_X(x) = \int_{-\infty}^x f_X(u) du$.

Expectation and variance.

$$\mathbb{E}[X] = \sum_x x p_X(x) \quad (\text{discrete}), \quad \mathbb{E}[X] = \int_{-\infty}^{\infty} x f_X(x) dx \quad (\text{continuous}),$$

$$\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2.$$

Expectation is *linear*: $\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y]$ for any random variables X, Y and constants a, b — this single fact is what lets us pull the recursive Bellman equations apart in Chapters 3 and 4.

1.1.4 Multiple random variables

For a pair (X, Y) with joint PMF/PDF $p_{X,Y}$, the marginal is recovered by summing/integrating out the other variable, e.g. $p_X(x) = \sum_y p_{X,Y}(x, y)$. The *conditional expectation* $\mathbb{E}[Y | X = x]$ is the mean of Y under the conditional distribution $p_{Y|X}(\cdot | x)$, and satisfies the *tower property*

$$\mathbb{E}[Y] = \mathbb{E}_X[\mathbb{E}[Y | X]],$$

which is the exact identity used repeatedly to unroll returns: we condition on the next state/action and then average over it.

Two random variables are independent iff $p_{X,Y}(x, y) = p_X(x)p_Y(y)$ for all x, y . $X_1, \dots, X_n$ possess the *Markov property* with respect to time if $p(x_{t+1} | x_1, \dots, x_t) = p(x_{t+1} | x_t)$ — this is the defining property of every model in Chapter 3 onward.

1.2 Linear Algebra

1.2.1 Matrices and basic operations

For $A \in \mathbb{R}^{m \times n}$, the transpose $A^\top \in \mathbb{R}^{n \times m}$ satisfies $(A^\top)_{ij} = A_{ji}$. A is *symmetric* if $A = A^\top$. The *identity* I_n has 1's on the diagonal, 0's elsewhere, and satisfies $AI = IA = A$. A matrix A is *invertible* if there exists A^{-1} with $AA^{-1} = A^{-1}A = I$; this exists iff $\det(A) \neq 0$ iff the columns of A are linearly independent (full rank).

1.2.2 Norms

For $x \in \mathbb{R}^n$, the standard norms used in convergence proofs are

$$\|x\|_1 = \sum_i |x_i|, \quad \|x\|_2 = \sqrt{\sum_i x_i^2}, \quad \|x\|_\infty = \max_i |x_i|.$$

Key Idea 1.2.1: Why the max-norm matters

Convergence of Dynamic Programming (Chapter 5) is proved by showing the Bellman backup is a *contraction* in the max-norm $\|v\|_\infty = \max_s |v(s)|$: shrinking the largest error at any single state is enough to guarantee the whole vector converges. Keep this norm in mind for Theorem 5.2.2.

1.2.3 Eigenvalues, positive semi-definiteness

λ is an *eigenvalue* of square A with eigenvector $x \neq 0$ if $Ax = \lambda x$. A symmetric matrix A is *positive semi-definite* (PSD) if $x^\top Ax \geq 0$ for all x ; equivalently, all eigenvalues of A are ≥ 0 . PSD-ness of the Hessian of an objective is what certifies that a stationary point of a quadratic loss (as in linear value function approximation, Chapter 11) is a global minimum, not a saddle.

1.2.4 Matrix calculus: gradients used later

For $w \in \mathbb{R}^n$, $\nabla_w J(w) \in \mathbb{R}^n$ collects the partial derivatives $(\nabla_w J)_i = \partial J / \partial w_i$. Two identities used repeatedly in Chapter 11:

$$\nabla_w (x^\top w) = x, \quad \nabla_w [(y - x^\top w)^2] = -2(y - x^\top w)x,$$

the second obtained by the chain rule on the square. These two lines are the entire content behind every gradient-descent value-function update you will see later — everything else is a choice of what to plug in for the “target” y .

1.2.5 Solving linear systems: why $(I - \gamma P)^{-1}$ appears

Given $A \in \mathbb{R}^{n \times n}$ invertible and $b \in \mathbb{R}^n$, the linear system $Ax = b$ has the unique solution $x = A^{-1}b$. This is a direct, $O(n^3)$ -cost way to solve for a value function once we express the Bellman equation in matrix form (Section 3.3.3); for large n we will instead use the iterative methods of Chapter 5.

Chapter 2

Introduction to Reinforcement Learning

Reinforcement learning (RL) is the study of how an agent should act in an environment in order to maximise cumulative reward. This chapter fixes the vocabulary — agent, environment, reward, history, state, policy, value function, model — that every later chapter builds on.

2.1 What is Reinforcement Learning?

RL sits at the intersection of many fields (control theory, operations research, psychology, neuroscience, economics) and is distinguished from supervised/unsupervised learning by a few defining features:

- There is no supervisor, only a *reward signal*.
- Feedback may be *delayed*, not instantaneous.
- Time really matters: data is sequential, non-i.i.d.
- The agent's actions affect the subsequent data it receives (the agent is *not* a passive observer).

Definition 2.1.1: Reward Hypothesis

All goals can be described by the maximisation of expected cumulative reward. Formally, at each time-step t the environment emits a scalar reward $R_t \in \mathbb{R}$, and the agent's objective is to select actions so as to maximise $\mathbb{E} \left[\sum_t R_t \right]$ (discounted or otherwise, see Chapter 3).

Worked Example 2.1.1: Reward design across domains

- **Financial investment:** actions have long-term consequences; reward may be delayed; it can be optimal to sacrifice immediate reward for larger future reward.
- **Robot motion planning:** explore the environment to learn optimal or near-optimal movement.
- **Autonomous driving:** reward could combine accident avoidance, fuel efficiency, and journey time — designing this combination is itself a hard problem.
- **Healthcare:** personalised dose control, hospital resource management, treatment recommendation.

2.2 The Agent–Environment Interface

Formally, at each step t :

- **The agent** executes action A_t , receives observation O_t , receives scalar reward R_t .

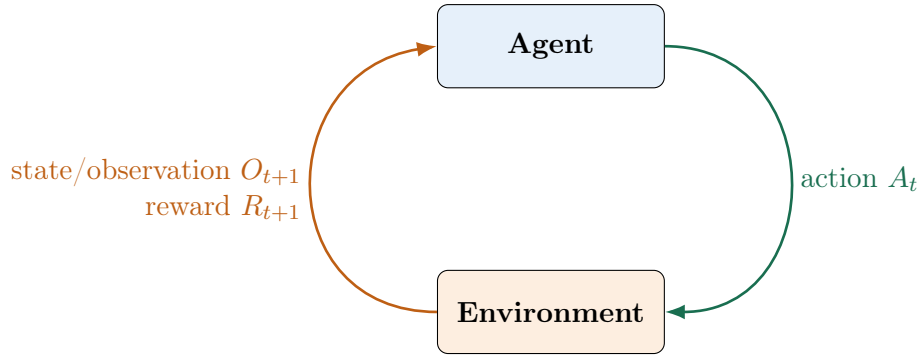


Figure 2.1: The agent–environment loop. At each step t the agent executes A_t ; the environment emits observation O_{t+1} and scalar reward R_{t+1} ; t is incremented at the environment step.

- **The environment** receives A_t , emits observation O_{t+1} , emits scalar reward R_{t+1} .

2.2.1 History and state

Definition 2.2.1: History

The history is the sequence of all observable variables up to time t :

$$H_t = O_1, R_1, A_1, \dots, A_{t-1}, O_t, R_t.$$

Whatever happens next depends on the history: the agent selects actions based on it, and the environment selects observations/rewards based on it (and the action just taken).

Definition 2.2.2: State

State is the information used to determine what happens next; formally it is any function of the history, $S_t = f(H_t)$.

Definition 2.2.3: Information (Markov) State

An information state contains all useful information from the history:

$$\mathbb{P}[S_{t+1} \mid S_t] = \mathbb{P}[S_{t+1} \mid S_1, \dots, S_t],$$

i.e. “the future is independent of the past given the present.” Once the state is known, the history may be discarded — S_t is a sufficient statistic of the future, $H_{1:t} \rightarrow S_t \rightarrow H_{t+1:\infty}$.

Remark. Note that H_t is trivially Markov (with respect to itself), and the true environment state S_t^e is always Markov by construction. The interesting question is whether the agent’s own constructed state S_t^a is Markov.

2.2.2 Full vs. partial observability

- **Full observability:** the agent directly observes the environment state, $O_t = S_t^a = S_t^e$. Agent state = environment state = information state. This is formally a *Markov Decision Process* (MDP, Chapter 4).
- **Partial observability:** the agent indirectly observes the environment (e.g. a robot with camera vision that is not told its absolute location; a trading agent that only observes current prices; a poker agent that only sees public cards). Now $S_t^a \neq S_t^e$, and the agent must construct its own state representation — e.g. the complete history $S_t^a = H_t$, a belief

over environment states, or a recurrent neural-network state. This is formally a *Partially Observable MDP* (POMDP).

2.3 Inside an RL Agent

An RL agent may include one or more of the following components.

Definition 2.3.1: Policy

A policy π is the agent's behaviour function, a map from state to action.

- Deterministic policy: $a = \pi(s)$.
- Stochastic policy: $\pi(a | s) = \mathbb{P}[A_t = a | S_t = s]$.

Definition 2.3.2: Value Function

A value function is a prediction of expected future reward, used to evaluate the goodness/badness of states (and hence to select between actions). We will define it precisely (with a discount factor) in Chapter 3.

Definition 2.3.3: Model

A model predicts what the environment will do next: $\mathcal{P}$ predicts the next state, $\mathcal{R}$ predicts the next immediate reward.

2.3.1 Taxonomy of RL agents

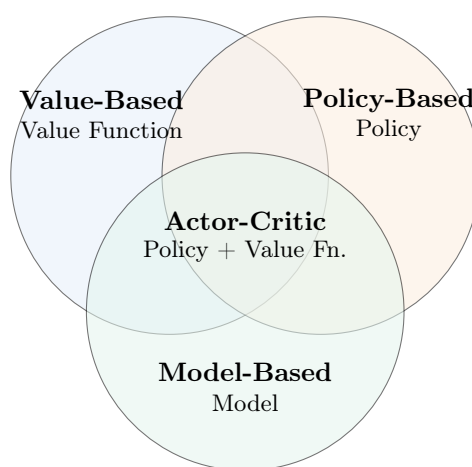


Figure 2.2: RL agent taxonomy. Orthogonally, agents are *model-free* (learn policy/value directly from experience, e.g. Chapters 6–10) or *model-based* (learn/use a model of $\mathcal{P}, \mathcal{R}$ and plan, e.g. Chapter 5).

2.4 Learning and Planning, Prediction and Control

Two fundamental problems in sequential decision making:

- **Reinforcement Learning:** the environment is initially unknown; the agent interacts with the environment and improves its policy from experience.
- **Planning:** a model of the environment is known; the agent performs computation with its model (no external interaction) and improves its policy — also called deliberation, reasoning, search.

Within either setting we distinguish two sub-problems that recur in every chapter below:

- **Prediction** (a.k.a. policy evaluation): evaluate the future, given a fixed policy π . *"How much reward will I get if I follow π ?"*
- **Control**: optimise the future, i.e. find the best policy. *"What is the best I can possibly do?"*

Control is almost always solved *via* repeated prediction — this is the Generalised Policy Iteration idea that threads through Chapters 5, 8, and 9.

2.5 Exploration and Exploitation

Key Idea 2.5.1: The Explore/Exploit Dilemma

RL is like trial-and-error learning: the agent should discover a good policy from its experience of the environment, without losing too much reward along the way.

- **Exploitation** makes the best decision given current information.
- **Exploration** gathers more information about the environment.

The best long-term strategy may involve short-term sacrifices, and it is usually necessary to do both — pure exploitation risks getting stuck in a sub-optimal policy; pure exploration wastes reward. This tension resurfaces concretely as ϵ -greedy policies in Chapter 8.

Exam Tip 2.5.1: What examiners like to ask here

Be ready to (i) state the reward hypothesis precisely, (ii) write down H_t and $S_t = f(H_t)$ and explain the Markov property in words *and* symbols, (iii) classify a given scenario as fully or partially observable and justify it, and (iv) place a described agent correctly on the value-based / policy-based / actor-critic and model-free / model-based axes.

Chapter 3

Markov Processes and Markov Reward Processes

We now build up the mathematical model of “the world” one piece at a time: first a bare Markov chain (states only), then a Markov Reward Process (states + rewards), then in Chapter 4 a full Markov Decision Process (states + rewards + actions). Each layer’s value function and Bellman equation is a special case of the next chapter’s, so understanding this chapter thoroughly makes Chapter 4 almost free.

3.1 The Markov Property

Definition 3.1.1: Markov Property

A state S_t is Markov if and only if

$$\mathbb{P}[S_{t+1} \mid S_t] = \mathbb{P}[S_{t+1} \mid S_1, \dots, S_t].$$

In words: the future is independent of the past given the present. Once the state is known, the history can be thrown away — the state alone is a sufficient statistic for predicting the future.

Key Idea 3.1.1: Stochastic processes, informally

A stochastic process is a collection of random variables indexed by time; it defines a probability distribution over *paths* (trajectories) that describe how some quantity evolves over time. A canonical example is the simple random walk $X_k = \sum_{i=1}^k \xi_i$ with i.i.d. ± 1 steps ξ_i , which has (i) zero mean $\mathbb{E}[X_k] = 0$, (ii) independent increments, and (iii) stationary increments (the distribution of $X_{k+h} - X_k$ depends only on h). A Markov chain is the special case of a stochastic process where, in addition, only the *most recent* point of the trajectory affects what happens next.

3.2 Markov Chains

Definition 3.2.1: Markov Process (Markov Chain)

A Markov Process is a memoryless random process: a sequence of random states $S_1, S_2, \dots$ with the Markov property. It is a tuple $\langle \mathcal{S}, P \rangle$ where

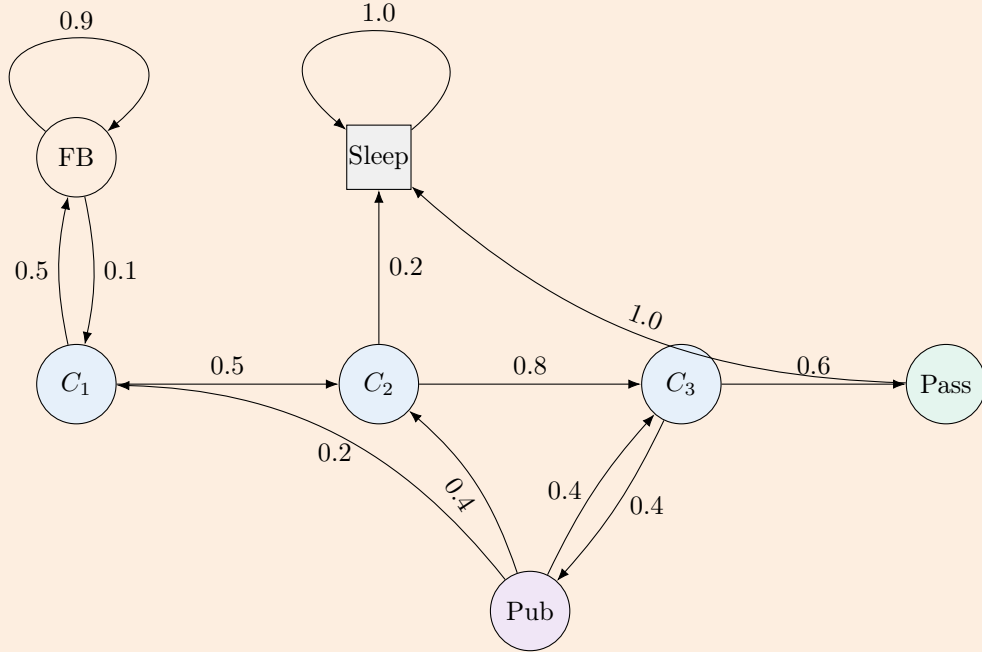
- $\mathcal{S}$ is a (finite) set of states,

- P is a state transition probability matrix,

$$P_{ss'} = \mathbb{P}[S_{t+1} = s' \mid S_t = s], \quad \sum_{s' \in \mathcal{S}} P_{ss'} = 1 \quad \forall s.$$

Equivalently, $\{X_0, X_1, X_2, \dots\}$ is a Markov chain if X_{t+1} depends only on X_t and not on $X_0, \dots, X_{t-1}$. A trajectory is one particular realised sequence $X_0, X_1, \dots$.

Worked Example 3.2.1: The Student Markov Chain



Reading off row **Class 1** of P : with probability 0.5 go to Class 2, with probability 0.5 go to Facebook. Sample episodes starting from C_1 (using Silver's indexing $C_1, C_2, C_3, \text{Pass}, \text{Sleep}, \text{FB}, \text{Pub}$):

- $C_1 C_2 C_3 \text{Pass Sleep}$
- $C_1 \text{FB FB } C_1 C_2 \text{Sleep}$
- $C_1 C_2 C_3 \text{Pub } C_2 C_3 \text{Pass Sleep}$
- $C_1 \text{FB FB } C_1 C_2 C_3 \text{Pub } C_1 \text{FB FB FB } C_1 C_2 C_3 \text{Pub } C_2 \text{Sleep}$

3.2.1 Multi-step transition probabilities

Proposition 3.2.1: n -step transitions

Let $P_{ss'}^{(n)} = \mathbb{P}[S_{t+n} = s' \mid S_t = s]$. Then

$$P^{(n)} = P^n \quad (\text{ordinary matrix power}).$$

Proof. By induction on n . The base case $n = 1$ is the definition of P . For the inductive step, condition on the intermediate state S_{t+1} and use the Markov property together with the Chapman–Kolmogorov identity:

$$\mathbb{P}[S_{t+n+1} = s' \mid S_t = s] = \sum_{u \in \mathcal{S}} \mathbb{P}[S_{t+1} = u \mid S_t = s] \mathbb{P}[S_{t+n+1} = s' \mid S_{t+1} = u] = \sum_u P_{su} P_{us'}^{(n)} = (P P^{(n)})_{ss'}.$$

By the inductive hypothesis $P^{(n)} = P^n$, so $P^{(n+1)} = P \cdot P^n = P^{n+1}$. ■

This answers precisely the natural question “what is the probability of going from state i to state j in exactly t steps?”: read off entry (i, j) of P^t .

3.3 Markov Reward Processes

Definition 3.3.1: Markov Reward Process (MRP)

A Markov Reward Process is a Markov chain with values: a tuple $\langle \mathcal{S}, P, R, \gamma \rangle$ where

- $\mathcal{S}$ is a finite set of states, $P_{ss'} = \mathbb{P}[S_{t+1} = s' \mid S_t = s]$;
- R is a reward function, $R_s = \mathbb{E}[R_{t+1} \mid S_t = s]$;
- $\gamma \in [0, 1]$ is a discount factor.

3.3.1 Return

Definition 3.3.2: Return

The return G_t is the total *discounted* reward from time-step t :

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

The *horizon* H is the number of time-steps in an episode (possibly ∞); if $H < \infty$ the process is a finite MRP.

Why discount at all?

- It is mathematically convenient: for $\gamma < 1$ and bounded rewards, G_t is a finite, well-defined random variable even for $H = \infty$ (a geometric series bound), avoiding infinite returns in cyclic Markov processes.
- It avoids infinite loops entirely dominating the objective.
- Uncertainty about the future may not be fully represented, and it is natural to trust immediate reward more than distant, uncertain reward.
- If the reward is financial, immediate rewards may literally earn more interest than delayed rewards.
- Animal/human behaviour empirically shows preference for immediate reward, i.e. an implicit discount factor < 1 .
- $\gamma = 0$: “myopic” evaluation, cares only about immediate reward. $\gamma = 1$: “far-sighted” evaluation, future reward is exactly as valuable as immediate reward. If every episode is guaranteed to terminate ($H < \infty$), $\gamma = 1$ is a legitimate (undiscounted) choice.

3.3.2 Value function

Definition 3.3.3: State-Value Function of an MRP

The state-value function $v(s)$ of an MRP is the expected return starting from state s :

$$v(s) = \mathbb{E}[G_t \mid S_t = s].$$

Worked Example 3.3.1: Student MRP: returns and value function

Attach rewards to the Student Markov chain: $R(\text{FB}) = -1$, $R(C_1) = -2$, $R(C_2) = -2$, $R(C_3) = -2$, $R(\text{Pub}) = +1$, $R(\text{Pass}) = +10$, $R(\text{Sleep}) = 0$ (a terminal, absorbing state). Take the sample trajectory $C_1, C_2, C_3, \text{Pass}, \text{Sleep}$ and compute G_1 (the return starting from the first C_1) for two discount factors:

$$\gamma = 0 : G_1 = -2 \quad (\text{only the immediate reward counts})$$

$$\gamma = \frac{1}{2} : G_1 = -2 - 2 \cdot \frac{1}{2} - 2 \cdot \frac{1}{4} + 10 \cdot \frac{1}{8} = -2 - 1 - 0.5 + 1.25 = -2.25$$

$$\gamma = 1 : G_1 = -2 - 2 - 2 + 10 = 4.$$

The value function $v(s)$ for $\gamma = 1$ is the *average* of G_t over all trajectories starting at s (in the limit of infinitely many sampled episodes) — this averaging-over-randomness is exactly what Monte-Carlo methods (Chapter 6) estimate directly, and what the Bellman equation below computes exactly and in closed form.

3.3.3 The Bellman Equation for MRPs

This is the single most important derivation in the whole course: everything from Dynamic Programming to Q-learning is some variant of unrolling this one-step recursion.

Theorem 3.3.1: Bellman Equation for MRPs

For any MRP $\langle \mathcal{S}, P, R, \gamma \rangle$, the value function satisfies

$$v(s) = \mathbb{E} [R_{t+1} + \gamma v(S_{t+1}) \mid S_t = s] = R_s + \gamma \sum_{s' \in \mathcal{S}} P_{ss'} v(s').$$

Proof. Start from the definition of the return and split off the first term:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \cdots) = R_{t+1} + \gamma G_{t+1}.$$

Take expectations conditional on $S_t = s$ and use linearity of expectation:

$$v(s) = \mathbb{E}[G_t \mid S_t = s] = \mathbb{E}[R_{t+1} \mid S_t = s] + \gamma \mathbb{E}[G_{t+1} \mid S_t = s].$$

The first term is R_s by definition. For the second term, apply the tower property, conditioning on S_{t+1} :

$$\mathbb{E}[G_{t+1} \mid S_t = s] = \mathbb{E} \left[\mathbb{E}[G_{t+1} \mid S_{t+1}] \mid S_t = s \right] = \mathbb{E} [v(S_{t+1}) \mid S_t = s],$$

where the inner equality $\mathbb{E}[G_{t+1} \mid S_{t+1}] = v(S_{t+1})$ holds because G_{t+1} depends on the future only through S_{t+1} — this is exactly where the *Markov property* is used (without it, $\mathbb{E}[G_{t+1} \mid S_t = s]$ could depend on the whole history, not just on v evaluated at S_{t+1}). Expanding the outer expectation over the possible next states s' with probabilities $P_{ss'}$ gives the stated sum. ■ ■

Key Idea 3.3.1: Reading the Bellman equation

The value of a state decomposes into two parts: the *immediate reward* R_{t+1} , and the *discounted value of the successor state* $\gamma v(S_{t+1})$. This one-step look-ahead recursion is what makes value functions computable at all — rather than summing an infinite series directly, we only ever need to relate $v(s)$ to v at the (finitely many) neighbouring states.

3.3.4 Matrix form and the direct solution

The Bellman equation can be written for all states simultaneously as a single vector equation. Let $v = (v(1), \dots, v(n))^T$ and $R = (R_1, \dots, R_n)^T$:

$$v = R + \gamma P v.$$

Proposition 3.3.1: Direct solution of the Bellman equation

Provided $(I - \gamma P)$ is invertible,

$$v = (I - \gamma P)^{-1} R.$$

Proof. $v = R + \gamma P v \iff v - \gamma P v = R \iff (I - \gamma P)v = R$; left-multiply by $(I - \gamma P)^{-1}$. ■

Remark. Matrix inversion costs $O(n^3)$ for n states, which is only feasible for small MRPs (a few thousand states at most). This is precisely the computational motivation for the iterative algorithms of Chapter 5 (which cost $O(n^2)$ per iteration using only matrix-vector products), and, at even larger scale, for the sampling-based methods of Chapters 6–10 that never form P at all.

Exam Tip 3.3.1: Common exam trap

Students often forget that the Bellman equation for an MRP has *no max and no action* — there is nothing to optimise, since a bare MRP has no choices. The max only enters once we add actions in Chapter 4. Also: don't forget that R_s is itself an *expectation* $\mathbb{E}[R_{t+1} \mid S_t = s]$ — rewards can be stochastic.

Chapter 4

Markov Decision Processes

We add *actions* to the MRP of Chapter 3, giving the agent choices for the first time. Almost every definition below is the MRP definition with an extra conditioning on action A_t and an extra $\max_\pi$ or $\max_a$ once we start optimising. Nearly all of RL (planning, prediction, control) is this chapter's machinery being computed approximately, incrementally, or from samples.

4.1 Definition

Definition 4.1.1: Markov Decision Process

A Markov Decision Process is a Markov Reward Process with decisions: an environment in which *all* states are Markov. It is a tuple $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$ where

- $\mathcal{S}$ is a finite set of states, $\mathcal{A}$ a finite set of actions;
- $P_{ss'}^a = \mathbb{P}[S_{t+1} = s' \mid S_t = s, A_t = a]$;
- $R_s^a = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$;
- $\gamma \in [0, 1]$ is a discount factor.

4.2 Policies

Definition 4.2.1: Policy

A policy π is a distribution over actions given states,

$$\pi(a \mid s) = \mathbb{P}[A_t = a \mid S_t = s].$$

A policy fully defines the behaviour of an agent (MDP policies depend only on the current state, not on the full history — this is sufficient because the environment state is Markov). Given a fixed MDP $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$ and a fixed policy π :

Proposition 4.2.1: Induced MRP

The state sequence $S_1, S_2, \dots$ is a Markov process $\langle \mathcal{S}, P^\pi \rangle$, and the state and reward sequence $S_1, R_2, S_2, \dots$ is a Markov Reward Process $\langle \mathcal{S}, P^\pi, R^\pi, \gamma \rangle$ where

$$P_{ss'}^\pi = \sum_{a \in \mathcal{A}} \pi(a \mid s) P_{ss'}^a, \quad R_s^\pi = \sum_{a \in \mathcal{A}} \pi(a \mid s) R_s^a.$$

Proof. Marginalise the action out: $\mathbb{P}[S_{t+1} = s' \mid S_t = s] = \sum_a \mathbb{P}[A_t = a \mid S_t = s] \mathbb{P}[S_{t+1} = s' \mid S_t = s, A_t = a] = \sum_a \pi(a \mid s) P_{ss'}^a$, and likewise for the reward by the tower property,

$$\mathbb{E}[R_{t+1} \mid S_t = s] = \sum_a \pi(a \mid s) \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]. \quad \blacksquare$$

Key Idea 4.2.1: Why this matters

Every MDP quantity we care about, *for a fixed policy*, can be computed as the corresponding MRP quantity of the induced MRP $\langle \mathcal{S}, P^\pi, R^\pi, \gamma \rangle$. This is exactly the “Bellman Expectation Equation in matrix form” shortcut below, and it is why the DP algorithms of Chapter 5 for MRP prediction generalise immediately to MDP prediction.

4.3 Value Functions

Definition 4.3.1: State-Value and Action-Value Functions

The state-value function $v_\pi(s)$ of an MDP is the expected return starting from state s , then following policy π :

$$v_\pi(s) = \mathbb{E}_\pi [G_t \mid S_t = s].$$

The action-value function $q_\pi(s, a)$ is the expected return starting from state s , taking action a , and thereafter following π :

$$q_\pi(s, a) = \mathbb{E}_\pi [G_t \mid S_t = s, A_t = a].$$

v_π tells you how good it is to *be* in a state; q_π tells you how good it is to *do* an action from a state. The relationship between the two is the heart of the theory.

4.3.1 The Bellman Expectation Equation

Theorem 4.3.1: Bellman Expectation Equation (v_π in terms of q_π)

$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) q_\pi(s, a).$$

Proof. By the tower property, conditioning G_t 's expectation on the action taken: $v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s] = \sum_a \mathbb{P}[A_t = a \mid S_t = s] \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] = \sum_a \pi(a \mid s) q_\pi(s, a). \quad \blacksquare$

Theorem 4.3.2: Bellman Expectation Equation (q_π in terms of v_π)

$$q_\pi(s, a) = R_s^a + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a v_\pi(s').$$

Proof. Identical to the MRP Bellman-equation proof (Theorem 3.3.1) but now everything is additionally conditioned on $A_t = a$: split $G_t = R_{t+1} + \gamma G_{t+1}$, take $\mathbb{E}[\cdot \mid S_t = s, A_t = a]$, use linearity, then the tower property conditioning on S_{t+1} (Markov property removes any further dependence on the action once S_{t+1} is fixed, since the future is generated by π from that point). $\blacksquare$

Theorem 4.3.3: Bellman Expectation Equation for v_π (two-step form)

$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) \left(R_s^a + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a v_\pi(s') \right).$$

Proof. Substitute Theorem 4.3.2 into Theorem 4.3.1. $\blacksquare$

Theorem 4.3.4: Bellman Expectation Equation for q_π (two-step form)

$$q_\pi(s, a) = R_s^a + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a \sum_{a' \in \mathcal{A}} \pi(a' | s') q_\pi(s', a').$$

Proof. Substitute Theorem 4.3.1 (evaluated at s') into Theorem 4.3.2. ■

Key Idea 4.3.1: The two-step look-ahead trees

Theorem 4.3.3 says: from s , average over actions $a \sim \pi(\cdot | s)$ (a “•” branch), then for each a average over next states $s' \sim P_s^a$ (an “o” branch), picking up reward R_s^a along the way and discounting $v_\pi(s')$ by γ . This alternation of *action-choice* averaging and *environment-transition* averaging is the “backup diagram” you will see reproduced in every algorithm in this course (Figure 7.1).

4.3.2 Matrix form

Exactly as in Section 3.3.3, using the induced MRP $\langle \mathcal{S}, P^\pi, R^\pi, \gamma \rangle$ of Proposition 4.2.1:

$$v_\pi = R^\pi + \gamma P^\pi v_\pi \quad \implies \quad v_\pi = (I - \gamma P^\pi)^{-1} R^\pi.$$

Worked Example 4.3.1: Bellman Expectation Equation in the Student MDP

Give the student four actions from C_2 : **Study** (leads to C_3 , reward -2) and **Sleep** (leads to terminal, reward 0); under a policy that picks each with probability 0.5,

$$v_\pi(C_2) = 0.5(-2 + \gamma v_\pi(C_3)) + 0.5(0 + \gamma \cdot 0).$$

With $\gamma = 1$ and the (given) optimal-random-policy values $v_\pi(C_3) = -1.3$, this gives $v_\pi(C_2) = 0.5(-2 - 1.3) + 0 = -1.65$, matching the value shown on the course’s Student MDP diagram for the equiprobable random policy. The general recipe is always: (i) enumerate the actions available at s with their probabilities under π , (ii) for each action write its immediate reward plus discounted successor value, (iii) take the π -weighted average.

4.4 Optimal Value Functions**Definition 4.4.1: Optimal Value Functions**

The optimal state-value function is $v_*(s) = \max_\pi v_\pi(s)$; the optimal action-value function is $q_*(s, a) = \max_\pi q_\pi(s, a)$. v_*/q_* specify the best possible performance in the MDP: an MDP is “solved” once v_* (or q_*) is known.

4.4.1 Existence and uniqueness of an optimal policy

Define a partial ordering over policies: $\pi \geq \pi' \iff v_\pi(s) \geq v_{\pi'}(s) \quad \forall s \in \mathcal{S}$.

Theorem 4.4.1: Existence of an Optimal Policy

For any MDP:

- (i) There exists an optimal policy π_* that is better than or equal to all other policies, $\pi_* \geq \pi \quad \forall \pi$.
- (ii) All optimal policies achieve the optimal value function, $v_{\pi_*}(s) = v_*(s)$.
- (iii) All optimal policies achieve the optimal action-value function, $q_{\pi_*}(s, a) = q_*(s, a)$.

Remark. A full proof requires the machinery of Chapter 5 (in particular that the Bellman optimality operator is a contraction with a unique fixed point, Theorem 5.4.2); the existence of a deterministic optimal policy then follows constructively by acting greedily w.r.t. that fixed point, as shown next.

Proposition 4.4.1: Finding an optimal policy from q_*

An optimal (deterministic) policy can be found by maximising over $q_*(s, a)$:

$$\pi_*(a \mid s) = \begin{cases} 1 & \text{if } a = \arg \max_{a \in \mathcal{A}} q_*(s, a) \\ 0 & \text{otherwise.} \end{cases}$$

There is always a deterministic optimal policy for any MDP, and if $q_*(s, a)$ is known, the optimal policy is obtained immediately, with no further planning required.

4.4.2 The Bellman Optimality Equation

Theorem 4.4.2: Bellman Optimality Equation (v_* in terms of q_*)

$$v_*(s) = \max_{a \in \mathcal{A}} q_*(s, a).$$

Proof. $v_*(s) = \max_{\pi} v_{\pi}(s) = \max_{\pi} \sum_a \pi(a \mid s) q_{\pi}(s, a)$. For a fixed s , this expectation over $a \sim \pi(\cdot \mid s)$ is maximised (over all distributions $\pi(\cdot \mid s)$, including deterministic ones) by placing all probability mass on $\arg \max_a q_{\pi}(s, a)$ — and by Proposition 4.4.1 we may simultaneously choose the same maximising π to attain $q_*(s, a)$ at that a . Hence $\max_{\pi} \sum_a \pi(a \mid s) q_{\pi}(s, a) = \max_a q_*(s, a)$. ■

Theorem 4.4.3: Bellman Optimality Equation (q_* in terms of v_*)

$$q_*(s, a) = R_s^a + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a v_*(s').$$

Proof. The immediate reward and transition dynamics do not depend on which policy is followed after the first action, only on (s, a) themselves; once at s' , the best possible continuation achieves $v_*(s')$ by definition. Formally this is Theorem 4.3.2 evaluated at the optimal policy π_* together with part (ii)-(iii) of Theorem 4.4.1. ■

Theorem 4.4.4: Bellman Optimality Equation for v_* (two-step form)

$$v_*(s) = \max_{a \in \mathcal{A}} \left(R_s^a + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a v_*(s') \right).$$

Theorem 4.4.5: Bellman Optimality Equation for q_* (two-step form)

$$q_*(s, a) = R_s^a + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a \max_{a' \in \mathcal{A}} q_*(s', a').$$

Both follow by substitution exactly as in the expectation-equation case, replacing the π -average with a max.

Worked Example 4.4.1: Bellman Optimality in the Student MDP

At C_3 (reward +10 for Study→Pass, or +1 immediate then $q_*(\text{Pub})$ for Pub), with $\gamma = 1$:

$$v_*(C_3) = \max \left\{ \underbrace{10}_{\text{Study} \rightarrow \text{Pass, terminal}}, \underbrace{1 + 0.2 v_*(C_1) + 0.4 v_*(C_2) + 0.4 v_*(C_3)}_{\text{Pub}} \right\}.$$

Solving the resulting simultaneous fixed-point equations across all states (by substitution or, in general, by the iterative methods of Chapter 5) yields $v_*(C_1) = 6$, $v_*(C_2) = 8$, $v_*(C_3) = 10$, $v_*(\text{FB}) = 6$, $v_*(\text{Pub}) = 8.4$, and reading off the maximising action at each state gives the optimal policy: always Study.

Key Idea 4.4.1: Why v_* has no closed-form solution

The Bellman *Expectation* Equation is linear in v_π (fixed weights $\pi(a | s)$), so it can be solved directly via $v_\pi = (I - \gamma P^\pi)^{-1} R^\pi$. The Bellman *Optimality* Equation is **non-linear** because of the max operator — there is no matrix M with $v_* = Mv_* + c$ in general. Consequently there is (in general) no closed-form solution, and we require the iterative algorithms of the next chapter: Value Iteration, Policy Iteration, and — once the model is unknown — Sarsa and Q-learning.

Exam Tip 4.4.1: The four Bellman equations, side by side

Examiners love asking you to write all four from memory. Memorise the pattern, not the formulas: (1) Expectation- v uses $\sum_a \pi(a|s)$; (2) Expectation- q uses $\sum_{s'} P_{ss'}^a$; (3) Optimality- v replaces $\sum_a \pi(a|s)(\cdot)$ with $\max_a(\cdot)$; (4) Optimality- q replaces $\sum_{a'} \pi(a'|s')q_\pi(s', a')$ inside the expectation- q two-step form with $\max_{a'} q_*(s', a')$. Everything reduces to composing Theorem 4.3.1/Theorem 4.4.2 with Theorem 4.3.2/Theorem 4.4.3.

Chapter 5

Planning by Dynamic Programming

Dynamic Programming (DP) assumes *full knowledge* of the MDP $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$ and uses it for planning. Every algorithm here is the Bellman equation of Chapter 4, turned into an update rule and iterated until convergence. This is the model-based extreme of the model-free spectrum we will move along in later chapters.

5.1 Why Dynamic Programming Applies to MDPs

Dynamic programming is a general method for solving complex problems by breaking them into subproblems, solving the subproblems, and combining the solutions. It applies whenever a problem has:

1. **Optimal substructure:** the global optimum can be decomposed into optimal sub-solutions (formally: the Principle of Optimality, Theorem 5.4.1 below).
2. **Overlapping subproblems:** sub-solutions recur many times, so they can be cached and reused.

Key Idea 5.1.1: MDPs satisfy both

The Bellman equation (Theorems 4.3.3 and 4.4.4) gives an exact *recursive decomposition* of $v(s)$ into a one-step reward plus the value of successor states — this is optimal substructure. The value function $v(s)$ at every state is exactly the cached sub-solution that gets reused by every other state that can reach s — this is the overlapping-subproblem structure. DP is therefore not just *applicable* to MDPs, it is close to the canonical example of the technique.

DP is used for *planning* in an MDP, covering both of our two sub-problems:

- **Prediction:** input $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$ and policy π (equivalently the induced MRP $\langle \mathcal{S}, P^\pi, R^\pi, \gamma \rangle$); output the value function v_π .
- **Control:** input $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$; output the optimal value function v_* and an optimal policy π_* .

5.2 Iterative Policy Evaluation

Problem: evaluate a given policy π (compute v_π).

Solution: iterative application of the Bellman *expectation* backup, $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_\pi$, using *synchronous* backups: at each iteration $k+1$, for *all* states $s \in \mathcal{S}$ simultaneously, update $v_{k+1}(s)$

from $v_k(s')$ where s' ranges over successors of s :

$$v_{k+1}(s) = \sum_{a \in \mathcal{A}} \pi(a | s) \left(R_s^a + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a v_k(s') \right), \quad \text{i.e.} \quad v_{k+1} = R^\pi + \gamma P^\pi v_k.$$

This is nothing but Theorem 4.3.3 with the unknown $v_\pi(s')$ on the right replaced by the current estimate $v_k(s')$ — an instance of *fixed-point iteration*.

5.2.1 Convergence: the contraction-mapping argument

Definition 5.2.1: Contraction Mapping

An operator T on a normed vector space is a γ -contraction ($\gamma \in [0, 1)$) if $\|Tu - Tv\| \leq \gamma\|u - v\|$ for all u, v .

Theorem 5.2.1: Banach Fixed-Point Theorem

If T is a γ -contraction on a complete normed vector space $(X, \|\cdot\|)$, then T has a unique fixed point $x^* = Tx^*$, and for any $x_0 \in X$ the sequence $x_{k+1} = Tx_k$ converges to x^* geometrically: $\|x_k - x^*\| \leq \gamma^k \|x_0 - x^*\|$.

Theorem 5.2.2: The Bellman Expectation Backup is a Contraction

Define the Bellman expectation backup operator T^π on $\mathbb{R}^{|\mathcal{S}|}$ by $T^\pi v = R^\pi + \gamma P^\pi v$. Then, in the max-norm $\|v\|_\infty = \max_{s \in \mathcal{S}} |v(s)|$, T^π is a γ -contraction.

Proof. For any $u, v \in \mathbb{R}^{|\mathcal{S}|}$ and any state s ,

$$(T^\pi u)(s) - (T^\pi v)(s) = \gamma \sum_{s'} P_{ss'}^\pi (u(s') - v(s')).$$

Taking absolute values and using the triangle inequality,

$$|(T^\pi u)(s) - (T^\pi v)(s)| \leq \gamma \sum_{s'} P_{ss'}^\pi |u(s') - v(s')| \leq \gamma \sum_{s'} P_{ss'}^\pi \|u - v\|_\infty = \gamma \|u - v\|_\infty,$$

where the middle inequality bounds every term $|u(s') - v(s')|$ by the max over s' , and the last equality uses $\sum_{s'} P_{ss'}^\pi = 1$. Since this bound holds for *every* s , it holds for the max over s : $\|T^\pi u - T^\pi v\|_\infty \leq \gamma \|u - v\|_\infty$. ■

Corollary 5.2.1: Convergence of Iterative Policy Evaluation

$(\mathbb{R}^{|\mathcal{S}|}, \|\cdot\|_\infty)$ is complete, and v_π is by construction the (unique) fixed point of T^π (this is exactly the Bellman equation, Theorem 4.3.3). By Theorem 5.2.1 and Theorem 5.2.2, iterative policy evaluation $v_{k+1} = T^\pi v_k$ converges to v_π from *any* initial guess v_0 , at a geometric rate $\|v_k - v_\pi\|_\infty \leq \gamma^k \|v_0 - v_\pi\|_\infty$.

Remark. *This is why DP algorithms are typically initialised at $v_0 \equiv 0$: the starting point is provably irrelevant to whether we converge, only to how fast.*

Worked Example 5.2.1: Iterative Policy Evaluation in the Small Gridworld

A 4×4 grid, states $1, \dots, 14$ non-terminal, one terminal state (shown top-left and bottom-right), reward -1 on every transition, actions leading off the grid leave the state unchanged, undiscounted ($\gamma = 1$), agent follows the equiprobable random policy $\pi(n|\cdot) = \pi(e|\cdot) =$

$\pi(s|\cdot) = \pi(w|\cdot) = 0.25$. Applying the synchronous backup above:

$k = 0$	0.0	0.0	0.0	0.0	$k = 1$	0.0	-1.0	-1.0	-1.0
	0.0	0.0	0.0	0.0		-1.0	-1.0	-1.0	-1.0
	0.0	0.0	0.0	0.0		-1.0	-1.0	-1.0	-1.0
	0.0	0.0	0.0	0.0		-1.0	-1.0	-1.0	0.0
$k = 2$	0.0	-1.7	-2.0	-2.0	$k = 3$	0.0	-2.4	-2.9	-3.0
	-1.7	-2.0	-2.0	-2.0		-2.4	-2.9	-3.0	-2.9
	-2.0	-2.0	-2.0	-1.7		-2.9	-3.0	-2.9	-2.4
	-2.0	-2.0	-1.7	0.0		-3.0	-2.9	-2.4	0.0
$k = 10$	0.0	-6.1	-8.4	-9.0	$k = \infty$	0.0	-14	-20	-22
	-6.1	-7.7	-8.4	-8.4		-14	-18	-20	-20
	-8.4	-8.4	-7.7	-6.1		-20	-20	-18	-14
	-9.0	-8.4	-6.1	0.0		-22	-20	-14	0.0

Sample computation, top-right corner at $k = 1$ from $k = 0$: from that state, all four actions incur reward -1 and land on a state with $v_0 = 0$, so $v_1 = 0.25 \times 4 \times (-1 + 0) = -1.0$, matching the table.

Key Idea 5.2.1: The greedy-policy bonus

Strikingly, acting *greedily* with respect to v_k already gives the *optimal* policy by $k = 3$ in this example — long before v_k itself has converged to v_π (which needs $k \rightarrow \infty$). This observation — that we often don't need a converged value function to extract a good/optimal policy — is exactly what motivates Value Iteration and truncated Policy Iteration below.

5.3 Policy Iteration

Given a policy π :

1. **Evaluate** π : $v_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma R_{t+2} + \dots | S_t = s]$ (Section 5.2).
2. **Improve** π by acting greedily with respect to v_π : $\pi' = \text{greedy}(v_\pi)$.

This process of policy iteration always converges to π_* , in a finite MDP, in a finite number of iterations (since there are finitely many deterministic policies and, as we prove next, each iteration either strictly improves the policy or has already reached optimality).

5.3.1 The Policy Improvement Theorem

Consider a deterministic policy $a = \pi(s)$, improved greedily:

$$\pi'(s) = \arg \max_{a \in \mathcal{A}} q_\pi(s, a).$$

Theorem 5.3.1: Policy Improvement

For any two policies π, π' related by $\pi'(s) = \arg \max_a q_\pi(s, a)$, $q_\pi(s, \pi'(s)) \geq v_\pi(s)$ for all s , and consequently $v_{\pi'}(s) \geq v_\pi(s)$ for all $s \in \mathcal{S}$.

Proof. *Step 1 (one-step improvement).* By construction of π' as an argmax and Theorem 4.3.1,

$$q_\pi(s, \pi'(s)) = \max_{a \in \mathcal{A}} q_\pi(s, a) \geq q_\pi(s, \pi(s)) = v_\pi(s).$$

Step 2 (propagate the improvement to all future steps). Starting from $v_\pi(s) \leq q_\pi(s, \pi'(s))$, unroll $q_\pi(s, \pi'(s))$ using Theorem 4.3.2 (an expectation now over the environment's transition *under action* $\pi'(s)$, which is exactly what "following π' for one step" means):

$$v_\pi(s) \leq q_\pi(s, \pi'(s)) = \mathbb{E}_{\pi'} [R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s].$$

Now apply Step 1 again, this time to the state S_{t+1} reached, i.e. substitute $v_\pi(S_{t+1}) \leq q_\pi(S_{t+1}, \pi'(S_{t+1}))$ inside the expectation:

$$v_\pi(s) \leq \mathbb{E}_{\pi'} [R_{t+1} + \gamma q_\pi(S_{t+1}, \pi'(S_{t+1})) \mid S_t = s] = \mathbb{E}_{\pi'} [R_{t+1} + \gamma R_{t+2} + \gamma^2 v_\pi(S_{t+2}) \mid S_t = s].$$

Iterating this substitution indefinitely (formally, by induction on the horizon, with the tail term $\gamma^n v_\pi(S_{t+n}) \rightarrow 0$ under standard boundedness/discounting assumptions) replaces every occurrence of v_π by one more real reward step under π' :

$$v_\pi(s) \leq \mathbb{E}_{\pi'} [R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \mid S_t = s] = v_{\pi'}(s).$$

■

Theorem 5.3.2: Policy Improvement Stops Only at Optimality

If the improvement step produces no change, i.e.

$$q_\pi(s, \pi'(s)) = \max_{a \in \mathcal{A}} q_\pi(s, a) = q_\pi(s, \pi(s)) = v_\pi(s) \quad \forall s,$$

then $v_\pi(s) = \max_{a \in \mathcal{A}} q_\pi(s, a)$ for all s — which is exactly the Bellman Optimality Equation (Theorem 4.4.2, Theorem 4.4.4). Hence $v_\pi = v_*$ and π is an optimal policy.

Proof. The displayed identity is precisely $v_\pi(s) = \max_a q_\pi(s, a)$, which coincides with Theorem 4.4.2 once we note $q_\pi = q_*$ is forced by the identity holding at every state (a short induction/limit argument, or: v_π satisfying the Bellman *optimality* equation makes it a fixed point of the optimality operator T^* defined in Section 5.4, whose fixed point is unique and equals v_* by Theorem 5.4.2 below). Since $v_\pi = v_*$, by definition π is optimal. ■

Key Idea 5.3.1: The full picture

Combining Theorem 5.3.1 and Theorem 5.3.2: every round of policy iteration either strictly improves v_π at some state (and does not decrease it anywhere), or we have already found π_* . In a finite MDP with finitely many deterministic policies, this process must terminate at π_* in finitely many iterations.

Worked Example 5.3.1: Jack's Car Rental

Two locations, up to 20 cars each. **State:** number of cars at each location. **Action:** move up to 5 cars between locations overnight. **Reward:** \$10 per car rented (only if a car is available). Cars are returned and requested at each location as independent Poisson random variables: location 1 has average requests = 3, average returns = 3; location 2 has average requests = 4, average returns = 2 ($\mathbb{P}[n \text{ events}] = \frac{\lambda^n}{n!} e^{-\lambda}$). Starting from $\pi_0 =$ "never move any cars," policy iteration alternates:

- **Evaluate** π_k by solving the (now enormous, ~ 400 -state) linear system / iterating Section 5.2;
- **Improve** greedily: at each state, move whichever number of cars (from -5 to $+5$) maximises the one-step-lookahead $q_{\pi_k}(s, a)$.

Successive policies $\pi_1, \pi_2, \pi_3, \pi_4$ shift cars more conservatively between locations, converging

to the optimal policy $\pi_4 = \pi_*$, which proactively re-balances stock toward location 2 (higher demand) each night. This is a canonical example of a problem too large to solve with the direct matrix inverse of Proposition 3.3.1 but perfectly tractable with Section 5.2's $O(n^2)$ -per-iteration update.

5.3.2 Modified / Generalised Policy Iteration

Does policy evaluation need to run all the way to convergence ($v_k \rightarrow v_\pi$) before every improvement step? Not necessarily:

- Introduce a stopping condition, e.g. ε -convergence of the value function ($\|v_{k+1} - v_k\|_\infty < \varepsilon$);
- or simply stop after a fixed k iterations of iterative policy evaluation (in the Small Grid-world, $k = 3$ already sufficed to reach the optimal policy, as noted above);
- or update the policy after *every single* backup, i.e. $k = 1$ — this is exactly **Value Iteration** (Section 5.4).

Definition 5.3.1: Generalised Policy Iteration (GPI)

GPI is the general idea of interleaving *any* policy-evaluation process (not necessarily complete) with *any* policy-improvement process (not necessarily fully greedy). Nearly every algorithm in this course — DP, Monte-Carlo control, Sarsa, Q-learning — is an instance of GPI with a different choice of evaluation method and improvement rule. The two processes compete (evaluation makes v consistent with the current π ; improvement makes π greedy w.r.t. the current v , generally making v inconsistent again) yet jointly converge to (v_*, π_*) , the unique point where both are simultaneously satisfied.

5.4 Value Iteration

Theorem 5.4.1: Principle of Optimality

A policy $\pi(a|s)$ achieves the optimal value from state s , $v_\pi(s) = v_*(s)$, if and only if for *every* state s' reachable from s , π also achieves the optimal value from s' : $v_\pi(s') = v_*(s')$.

Equivalently: any optimal policy can be subdivided into an optimal first action A_* , followed by an optimal policy from the successor state S' . This principle is precisely what licenses a one-step-lookahead backup as *sufficient* for optimality — we never need to plan further ahead than one step, provided the values we bootstrap from are already optimal for their states.

5.4.1 Deterministic Value Iteration

If we already know the solution to subproblems $v_*(s')$ for all successors s' of s , then $v_*(s)$ is found by one Bellman-optimality backup:

$$v_*(s) \leftarrow \max_{a \in \mathcal{A}} \left(R_s^a + \gamma \sum_{s' \in \mathcal{S}} P_{ss'}^a v_*(s') \right).$$

Value Iteration applies this update to *every* state synchronously, starting from any v_0 (e.g. $v_0 \equiv 0$), *without* ever constructing an explicit intermediate policy:

$$v_{k+1}(s) = \max_{a \in \mathcal{A}} \left(R_s^a + \gamma \sum_{s'} P_{ss'}^a v_k(s') \right), \quad \forall s \in \mathcal{S}.$$

Worked Example 5.4.1: Shortest Path as Value Iteration

On a grid where every edge costs -1 (equivalently, minimise the number of steps to a goal cell), $v_0 \equiv 0$; after one synchronous backup, cells adjacent to the goal become -1 ; after two backups, cells two steps away become -2 ; and so on — value iteration here literally implements breadth-first shortest-path computation, converging in exactly the graph diameter's number of iterations.

5.4.2 Convergence of Value Iteration**Lemma 5.4.1: The max operator is 1-Lipschitz**

For any two functions $f, g : \mathcal{A} \rightarrow \mathbb{R}$, $|\max_a f(a) - \max_a g(a)| \leq \max_a |f(a) - g(a)|$.

Proof. By symmetry assume $\max_a f(a) \geq \max_a g(a)$; let $a^* = \arg \max_a f(a)$. Then $\max_a f(a) - \max_a g(a) = f(a^*) - \max_a g(a) \leq f(a^*) - g(a^*) \leq \max_a |f(a) - g(a)|$, since $g(a^*) \leq \max_a g(a)$. The case $\max_a g(a) \geq \max_a f(a)$ is symmetric, giving the two-sided (absolute value) bound. ■

Theorem 5.4.2: The Bellman Optimality Backup is a Contraction

Define $T^*v = \max_a (R^a + \gamma P^a v)$ (i.e. $(T^*v)(s) = \max_a [R_s^a + \gamma \sum_{s'} P_{ss'}^a v(s')]$). Then T^* is a γ -contraction in the max-norm.

Proof. Fix s and write $f(a) = R_s^a + \gamma \sum_{s'} P_{ss'}^a u(s')$, $g(a) = R_s^a + \gamma \sum_{s'} P_{ss'}^a v(s')$. By Lemma 5.4.1,

$$|(T^*u)(s) - (T^*v)(s)| = |\max_a f(a) - \max_a g(a)| \leq \max_a |f(a) - g(a)| = \gamma \max_a \left| \sum_{s'} P_{ss'}^a (u(s') - v(s')) \right| \leq \gamma \|u - v\|_\infty,$$

by exactly the same averaging argument as in Theorem 5.2.2 (each $P_{s'}^a$ is a probability distribution). This holds for every s , so $\|T^*u - T^*v\|_\infty \leq \gamma \|u - v\|_\infty$. ■

Corollary 5.4.1: Convergence of Value Iteration

By Theorem 5.2.1, T^* has a unique fixed point — which by construction is exactly v_* (this is the Bellman Optimality Equation, Theorem 4.4.4) — and value iteration $v_{k+1} = T^*v_k$ converges to v_* from any v_0 , at rate $\|v_k - v_*\|_\infty \leq \gamma^k \|v_0 - v_*\|_\infty$. This is also the missing piece of the existence proof for Theorem 4.4.1: v_* exists and is unique as this fixed point, and $\pi_* = \text{greedy}(v_*)$ (via Proposition 4.4.1, using q_* derived from v_* by Theorem 4.4.3) attains it.

Remark. Note that intermediate value functions v_k produced during Value Iteration generally do not correspond to the value function of any actual policy — unlike Policy Iteration, where every $v_k = v_{\pi_k}$ is meaningful in its own right. Value Iteration only guarantees that the greedy policy at convergence is optimal (though, as the Small Gridworld example showed, the greedy policy w.r.t. v_k is often optimal well before v_k itself converges).

5.5 Summary of Synchronous DP Algorithms**5.6 Asynchronous Dynamic Programming**

All algorithms above use *synchronous* backups: every state is backed up “simultaneously,” i.e. using only values from the *previous* sweep. Asynchronous DP instead backs up states individually, in any order, and can significantly reduce computation — it is guaranteed to converge

Problem	Bellman Equation Used	Algorithm
Prediction	Bellman Expectation Equation	Iterative Policy Evaluation
Control	Bellman Expectation Eq. + greedy improvement	Policy Iteration
Control	Bellman Optimality Equation	Value Iteration

Table 5.1: Algorithms are based on the state-value function v_π/v_* , each iteration costing $O(mn^2)$ for m actions and n states. The same algorithms apply, at cost $O(m^2n^2)$ per iteration, to the action-value function q_π/q_* instead.

to the same fixed point provided every state continues to be selected infinitely often. Three common schemes:

- **In-place DP:** for each state s , immediately overwrite $v(s) \leftarrow \max_a(R_s^a + \gamma \sum_{s'} P_{ss'}^a v(s'))$ using whatever values (old *or* already-updated-this-sweep) are currently stored — unlike synchronous DP, which needs two copies of v (old and new), this needs only one, and typically converges faster since it propagates information immediately.
- **Prioritised sweeping:** use the magnitude of the Bellman error $|\max_a(R_s^a + \gamma \sum_{s'} P_{ss'}^a v(s')) - v(s)|$ to prioritise which state to back up next (largest error first), maintained in a priority queue; requires knowledge of *reverse* dynamics (which states lead into s).
- **Real-time DP:** back up only states that the agent actually visits (simulating or executing trajectories), focusing computation on the relevant part of the state space and skipping states that are never reached under good policies.

Key Idea 5.6.1: Full-width backups and the curse of dimensionality

Every DP backup in this chapter is *full-width*: it looks at *every* successor state and *every* action, requiring the complete model P, R . For even moderately large problems this is prohibitively expensive — DP is effective for MDPs with millions of states, but $|\mathcal{S}|$ can easily grow exponentially in the number of state variables (the “curse of dimensionality”). Even a single full backup can be too expensive for large problems, let alone the many sweeps needed to converge. This — together with the requirement of a fully known model — is precisely the practical motivation for the *sample backups* of the next three chapters: replace the full expectation over successors with a single sampled transition, and never require P, R to be known at all.

Exam Tip 5.6.1: Frequently examined points in this chapter

- Be able to reproduce the contraction-mapping convergence proof for *both* T^π (Theorem 5.2.2) and T^* (Theorem 5.4.2) — they are 90% identical, differing only in whether you’re bounding an expectation over π or a max.
- Be able to reproduce the Policy Improvement Theorem’s telescoping proof (Theorem 5.3.1) line by line.
- Know the complexity of each method: direct solve $O(n^3)$, one DP sweep $O(mn^2)$, and that Value Iteration has *no policy* associated with intermediate iterates while Policy Iteration does.

Chapter 6

Model-Free Prediction: Monte-Carlo Learning

Status update. To solve a *known* MDP we plan by Dynamic Programming (Chapter 5). Starting now, we drop the assumption that P, R are known: to *estimate* the value function of an *unknown* MDP, we turn to model-free prediction. This chapter covers the simplest approach — Monte-Carlo (MC) — which estimates $v_\pi(s)$ literally by averaging observed returns, with no knowledge of dynamics at all.

6.1 Monte-Carlo Reinforcement Learning

Definition 6.1.1: Monte-Carlo Policy Evaluation

Monte-Carlo methods:

- learn directly from *complete episodes* of experience;
- are **model-free**: no knowledge of MDP transitions P or reward function R is required;
- learn from complete episodes only — **no bootstrapping**: an episode must run to termination before any update is made;
- use the simplest possible idea: value = mean return;
- can only be applied to *episodic* MDPs — all episodes must terminate.

The target quantity is the definition of $v_\pi(s)$ itself, $v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$: MC simply replaces the (unknown) expectation by an empirical average over sampled episodes, using the law of large numbers to guarantee convergence as the number of episodes grows.

6.1.1 First-Visit and Every-Visit MC

Goal: learn v_π from episodes of experience under policy π , $S_1, A_1, R_2, \dots, S_k \sim \pi$.

Algorithm 1: First-Visit Monte-Carlo Policy Evaluation

```
1 for each episode do
2   for the first time-step  $t$  that state  $s$  is visited in the episode do
3     Increment counter  $N(s) \leftarrow N(s) + 1$ ;
4     Increment total return  $S(s) \leftarrow S(s) + G_t$ ;
5     Update estimate  $V(s) \leftarrow S(s)/N(s)$ ;
6 By the law of large numbers,  $V(s) \rightarrow v_\pi(s)$  as  $N(s) \rightarrow \infty$ .
```

Algorithm 2: Every-Visit Monte-Carlo Policy Evaluation

```

1 for each episode do
2   for every time-step  $t$  that state  $s$  is visited in the episode do
3     Increment counter  $N(s) \leftarrow N(s) + 1$ ;
4     Increment total return  $S(s) \leftarrow S(s) + G_t$ ;
5     Update estimate  $V(s) \leftarrow S(s)/N(s)$ ;
6 Also converges to  $v_\pi(s)$  (with different, generally lower, statistical efficiency properties
   than first-visit, since successive visits within the same episode are correlated).

```

Worked Example 6.1.1: Blackjack via Monte-Carlo

States (200 of them): current sum (12–21; below 12 the agent always draws automatically, so it's not a decision state), dealer's showing card (ace–10), whether the player holds a usable ace (yes/no). Actions: **stick** (stop, terminate) or **twist** (take another card, no replacement). Reward for **stick**: +1 if the player's final sum exceeds the dealer's, 0 on a draw, −1 if less. Reward for **twist**: −1 if the sum exceeds 21 (bust, terminate immediately), 0 otherwise. There is no discounting ($\gamma = 1$) and no explicit transition model is ever constructed — we only ever need to *sample* hands (easy) rather than enumerate P (awkward, given the dealer's fixed strategy). Running first-visit MC under the fixed policy “stick if sum ≥ 20 , else twist” for many simulated hands and averaging the returns per state produces a value surface that is low near sum = 12 and rises sharply near sum = 20–21, exactly matching intuition.

6.2 Incremental Mean and Incremental Monte-Carlo Updates

Storing every return G_t and recomputing $S(s)/N(s)$ from scratch is wasteful; instead we update the mean incrementally.

Proposition 6.2.1: Incremental Mean

The mean $\mu_1, \mu_2, \dots$ of a sequence $x_1, x_2, \dots$ can be computed incrementally:

$$\mu_k = \mu_{k-1} + \frac{1}{k}(x_k - \mu_{k-1}).$$

Proof. By definition $\mu_k = \frac{1}{k} \sum_{j=1}^k x_j$. Split off the last term:

$$\mu_k = \frac{1}{k} \left(x_k + \sum_{j=1}^{k-1} x_j \right) = \frac{1}{k} (x_k + (k-1)\mu_{k-1}) = \frac{1}{k} (x_k + k\mu_{k-1} - \mu_{k-1}) = \mu_{k-1} + \frac{1}{k} (x_k - \mu_{k-1}).$$

■

This is an “error-correction” form: the new estimate is the old estimate, nudged toward the newest sample x_k by a step of size $1/k$ proportional to the current *error* $x_k - \mu_{k-1}$.

Definition 6.2.1: Incremental Monte-Carlo Updates

Applying Proposition 6.2.1 to value-function estimation: after each episode, for each state S_t visited with return G_t ,

$$N(S_t) \leftarrow N(S_t) + 1, \quad V(S_t) \leftarrow V(S_t) + \frac{1}{N(S_t)} (G_t - V(S_t)).$$

In non-stationary problems (e.g. if the policy π generating the episodes is itself slowly changing, as it will be under MC *control* in Chapter 8), it is often better to use a fixed step-size

α instead of the exactly-averaging $1/N(S_t)$:

$$V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t)),$$

which *forgets* old episodes geometrically and tracks a moving target.

Key Idea 6.2.1: The general template

Every update rule in the rest of this course has the same skeleton:

$$\text{estimate} \leftarrow \text{estimate} + \alpha(\text{target} - \text{estimate}).$$

For MC, target = G_t , the actual (noisy, complete-episode) return. Chapter 7 changes only the *target*, replacing G_t by a bootstrapped one-step estimate — everything else in this template stays identical.

Exam Tip 6.2.1: First-visit vs. every-visit

Both are unbiased-in-the-limit and both converge to v_π , so an exam question contrasting them is really asking about statistical properties, not correctness: first-visit MC produces an average of *independent* samples across episodes (one G_t per episode per state), which is the version with the standard textbook unbiasedness/consistency proof via i.i.d. samples; every-visit MC can use multiple, *correlated* returns from within a single episode, which is data-efficient but complicates the classical i.i.d. analysis (it is still consistent, just via a different argument).

Chapter 7

Temporal-Difference Learning and TD(λ)

Temporal-Difference (TD) learning is the central idea of modern reinforcement learning: like Monte-Carlo, it learns directly from raw experience with no model; like Dynamic Programming, it *bootstraps* — it updates estimates based partly on other learned estimates, without waiting for a final outcome. This chapter builds TD(0), analyses its bias/variance trade-off against MC, and then unifies the two into a single family, TD(λ), via n -step returns and eligibility traces.

7.1 TD(0): the Simplest Temporal-Difference Method

Definition 7.1.1: TD(0)

TD methods learn directly from episodes of experience; are model-free (no knowledge of P, R); and — unlike MC — learn from *incomplete* episodes, by *bootstrapping*. The simplest TD algorithm, TD(0), updates $V(S_t)$ toward an *estimated* return:

$$V(S_t) \leftarrow V(S_t) + \alpha(R_{t+1} + \gamma V(S_{t+1}) - V(S_t)).$$

The quantity $\delta_t := R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ is called the **TD error**. $R_{t+1} + \gamma V(S_{t+1})$ is called the **TD target**.

Key Idea 7.1.1: Where the update comes from

Compare with the incremental MC update of Definition 6.2.1: MC uses target G_t ; TD(0) uses target $R_{t+1} + \gamma V(S_{t+1})$. This target is exactly the right-hand side of the (sample) Bellman equation Theorem 3.3.1, $v(s) = \mathbb{E}[R_{t+1} + \gamma v(S_{t+1}) \mid S_t = s]$, with the expectation replaced by a single sample transition and the unknown $v(S_{t+1})$ replaced by the current estimate $V(S_{t+1})$. TD(0) is therefore literally “turn the Bellman equation into a stochastic, one-sample update rule,” exactly as DP turned it into a full-expectation update rule in Section 5.2 — the difference is DP averages over *all* successors using known P ; TD(0) uses a single *sampled* successor and needs no model at all. TD “updates a guess towards a guess.”

Worked Example 7.1.1: Driving Home

You leave the office, and at each stage of the journey you have an estimate of the total time to get home; as new information arrives (it starts to rain, you exit onto the highway, you get stuck behind a truck) you don’t wait until you’re home to revise the estimate — you update your prediction *immediately*, based on the new situation’s predicted remaining time. E.g. leaving office (predicted total 30min); reach car, raining (predicted total revised to 35min); exit highway (predicted total 35min, no change); stuck behind truck (40min); ... arrive

home (actual 43min). TD updates each of these intermediate predictions toward the *next* prediction plus the elapsed time, exactly like $V(S_t) \leftarrow V(S_t) + \alpha(R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$; MC would instead wait until arriving home and then distribute the credit/blame for the entire 43-minute outcome across every earlier prediction equally.

7.1.1 Advantages and disadvantages of MC vs. TD

- TD can learn *before* knowing the final outcome; TD can learn online, after every single step; MC must wait until the end of an episode before the return is known.
- TD can learn without the final outcome at all — from *incomplete* sequences (useful in continuing, non-terminating environments); MC can only learn from *complete* sequences, and only works in episodic (terminating) environments.

7.1.2 Bias/Variance Trade-off

Proposition 7.1.1: Unbiasedness of the MC and “true” TD targets

The return G_t is, by definition, an unbiased estimate of $v_\pi(S_t)$: $\mathbb{E}_\pi[G_t | S_t] = v_\pi(S_t)$. The *true* TD target $R_{t+1} + \gamma v_\pi(S_{t+1})$ is also an unbiased estimate of $v_\pi(S_t)$: this is exactly Theorem 3.3.1/Theorem 4.3.3, taking expectation over S_{t+1}, R_{t+1} . However, the *actual* TD target used in practice, $R_{t+1} + \gamma V(S_{t+1})$, is a **biased** estimate of $v_\pi(S_t)$ whenever the current estimate $V \neq v_\pi$ (which is always, until convergence) — since $\mathbb{E}[R_{t+1} + \gamma V(S_{t+1}) | S_t] = R_{S_t} + \gamma \sum_{s'} P_{S_t s'} V(s') \neq R_{S_t} + \gamma \sum_{s'} P_{S_t s'} v_\pi(s') = v_\pi(S_t)$ in general.

Proposition 7.1.2: Variance comparison

The TD target has much lower variance than the return: G_t is a function of *many* random actions, transitions, and rewards along the entire remainder of the episode (variance accumulates additively along the whole trajectory), whereas the TD target $R_{t+1} + \gamma V(S_{t+1})$ depends on only *one* random action, transition, and reward.

Monte-Carlo	Temporal-Difference
High variance, <i>zero</i> bias	Low variance, <i>some</i> bias
Good convergence properties (even with function approximation)	Usually more data-efficient than MC
Not very sensitive to initial value	TD(0) converges to $v_\pi(s)$ (not always with non-linear function approximation)
Very simple to understand and use	More sensitive to the initial value
Does not exploit the Markov property — often more effective in <i>non</i> -Markov environments	Exploits the Markov property — usually more efficient in genuinely Markov environments

Table 7.1: Monte-Carlo vs. TD — the classic exam comparison table.

Worked Example 7.1.2: Random Walk

A chain of states with two terminal ends, all-zero reward except +1 on reaching the right terminal, no discounting. Empirically, TD(0) reduces estimation error (RMS error against the true, closed-form values) faster than first-visit MC per episode, and reaches a lower asymptotic error for a suitably tuned constant step-size α — a direct experimental consequence of

Proposition 7.1.2's lower-variance targets.

7.2 Batch MC and TD: the AB Example

Both $V(s) \rightarrow v_\pi(s)$ as experience $\rightarrow \infty$, but with a *finite* batch of K episodes repeatedly replayed (“batch updating,” apply MC or TD(0) to the fixed batch of episodes until the value function converges), do MC and TD(0) converge to the *same* answer?

Worked Example 7.2.1: The AB Example

Two states A, B , no discounting, 8 episodes of experience:

$$(A, 0, B, 0), \quad (B, 1) \times 6, \quad (B, 0).$$

i.e. one episode visits A (reward 0), transitions to B (reward 0), terminates; six episodes start directly at B with reward 1, terminate; one episode starts at B with reward 0, terminates. What are $V(A), V(B)$?

$V(B)$ (**both methods agree**): B is visited 8 times total, with returns 0, 1, 1, 1, 1, 1, 1, 0 (undiscounted, one-step returns). Both MC and TD converge, by construction, to the sample mean:

$$V(B) = \frac{0 + 1 + 1 + 1 + 1 + 1 + 1 + 0}{8} = \frac{6}{8} = 0.75.$$

$V(A)$ (**MC and TD disagree**):

- **MC**: A is visited exactly once, with observed return $G = 0 + 0 = 0$. Batch MC always converges to the sample means of *observed returns*, so $V_{\text{MC}}(A) = 0$. Equivalently, MC finds the value function that minimises the sum-squared error on the observed returns $\sum_k (G_k - V(s_k))^2$ in the training data — and $V(A) = 0$ fits that one observation exactly.
- **TD(0)**: batch TD(0) converges to the solution of the **maximum-likelihood Markov model** implied by the data (*certainty equivalence*): build the empirical transition matrix and reward function that best explain the observed episodes, then compute the exact value function of *that* model. Here, A transitioned to B with reward 0 in 100% of observed instances, so the fitted model gives $\hat{P}(B | A) = 1$, $\hat{R}(A) = 0$, and therefore, using the ordinary Bellman equation on the fitted model,

$$V_{\text{TD}}(A) = \hat{R}(A) + \hat{P}(B | A) V(B) = 0 + 1 \times 0.75 = 0.75.$$

Theorem 7.2.1: Certainty Equivalence

Batch TD(0) converges to the solution of the maximum-likelihood Markov model $\langle \hat{\mathcal{S}}, \hat{P}, \hat{R}, \gamma \rangle$ that best fits the observed data (in the sense of maximum likelihood over the observed state-action-reward-state transitions). Batch MC converges instead to the value function minimising mean-squared error on the observed returns directly, with no model in between.

Key Idea 7.2.1: Why this explains the earlier table

TD exploits the Markov property (it implicitly builds and exploits a Markov model of the data, which is why it does well when the environment genuinely *is* Markov); MC ignores the Markov property entirely and directly optimises observed returns, which is why it remains valid — unlike naive TD — even in non-Markov environments, where no consistent transition model exists.

7.3 Unifying View: Backup Diagrams

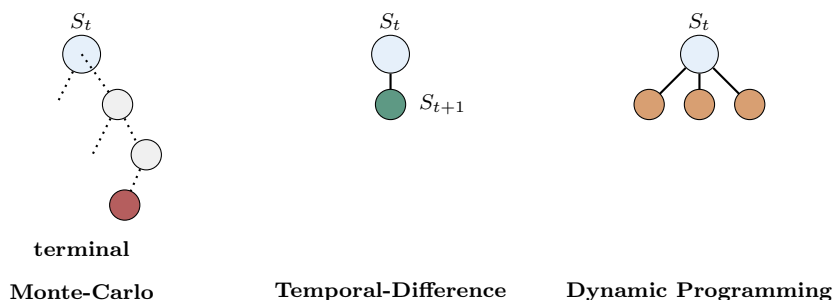


Figure 7.1: Backup diagrams. MC samples one full trajectory to termination (deep, narrow). DP considers every possible one-step successor exhaustively, weighted by P^π (shallow, full-width). TD samples a single one-step successor and bootstraps (shallow, narrow) — the best of both: a sample (like MC) combined with bootstrapping (like DP).

Definition 7.3.1: Bootstrapping and Sampling

Bootstrapping: the update uses an estimate, not an exact value.

- MC does *not* bootstrap (uses the actual return); DP bootstraps; TD bootstraps.

Sampling: the update samples an expectation rather than computing it exactly.

- MC samples; DP does *not* sample (full expectation via known P); TD samples.

This 2×2 classification (bootstrap? / sample?) is exactly what makes TD the “best of both worlds” cell of the table, and it is the organising principle behind every algorithm from here to the end of the course.

7.4 n -Step Prediction

TD(0) looks only one step ahead; MC looks all the way to termination. Between these extremes lies a whole family.

Definition 7.4.1: n -Step Return

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n}).$$

$n = 1$ recovers the TD(0) target; $n \rightarrow \infty$ (or $n = T - t$ for an episode of length T) recovers the MC return G_t .

The corresponding n -step **TD** update is $V(S_t) \leftarrow V(S_t) + \alpha(G_t^{(n)} - V(S_t))$.

Worked Example 7.4.1: Large Random Walk

On a long random-walk chain, sweeping n from 1 (pure TD(0)) to very large (near-MC) and plotting RMS error against the true value function shows a characteristic U-shape: performance is worst at both extremes and best for some *intermediate* n , for both online and offline updating. Neither pure TD(0) nor pure MC is best — this single empirical fact is the whole motivation for what follows.

7.4.1 Averaging n -step returns and the λ -return

We could average returns of different n (e.g. average the 2-step and 4-step returns) to capture the best of several n simultaneously — and in fact combine information from *all* n efficiently, using weight $(1 - \lambda)\lambda^{n-1}$ on the n -step return.

Definition 7.4.2: λ -Return

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}, \quad \lambda \in [0, 1].$$

Proposition 7.4.1: The weights form a valid convex combination

$$(1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} = 1.$$

Proof. Geometric series: $\sum_{n=1}^{\infty} \lambda^{n-1} = \frac{1}{1-\lambda}$ for $\lambda \in [0, 1)$, so $(1 - \lambda) \cdot \frac{1}{1-\lambda} = 1$. (For $\lambda = 1$ the weight is understood to collapse entirely onto $n \rightarrow \infty$, i.e. the MC return.) ■

Definition 7.4.3: Forward-View TD(λ)

$$V(S_t) \leftarrow V(S_t) + \alpha(G_t^\lambda - V(S_t)).$$

The weight given to the n -step return decays geometrically by a factor λ per extra step, giving the characteristic exponentially-decaying “TD(λ) weighting function.” Setting $\lambda = 0$ collapses all weight onto $n = 1$, recovering TD(0) exactly; letting $\lambda \rightarrow 1$ shifts all weight onto the full return, recovering (every-visit) MC.

Remark. *Forward-view TD(λ) is a theoretical construction: computing G_t^λ at time t requires knowing rewards arbitrarily far into the future, so — like MC — it can only be computed after an episode ends. It gives us the right target to aim for; the backward view below gives us a mechanism to compute (an offline-equivalent of) it online.*

7.5 Backward-View TD(λ): Eligibility Traces

Definition 7.5.1: Eligibility Trace

$$E_0(s) = 0, \quad E_t(s) = \gamma\lambda E_{t-1}(s) + \mathbf{1}(S_t = s).$$

Every time a state is visited, its trace is bumped up by 1; between visits, every state’s trace decays by the factor $\gamma\lambda$. The trace is a combined *frequency* heuristic (states visited more often get more credit/blame) and *recency* heuristic (states visited more recently get more credit/blame) for assigning credit to a TD error observed *now*.

Definition 7.5.2: Backward-View TD(λ)

Keep an eligibility trace for every state. At every step, compute the ordinary TD error and update *every* state in proportion to its current trace:

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t), \quad V(s) \leftarrow V(s) + \alpha \delta_t E_t(s) \quad \forall s \in \mathcal{S}.$$

This updates online, every step, from incomplete sequences — exactly the practical advantage TD(0) had over MC — while (as we prove below) matching the theoretically well-motivated forward-view target in total, over the course of an episode.

7.5.1 Two special cases

- $\lambda = 0$: only $E_t(S_t) = 1$ and all other traces are 0 at the moment of update (since a trace that hasn't been refreshed this step has already decayed to 0 by the $\gamma\lambda = 0$ factor), so the backward-view update reduces exactly to the TD(0) update of Definition 7.1.1.
- $\lambda = 1$: credit is deferred, undiscounted, all the way back to the start of the episode — we show next that the *total* update over an episode exactly matches every-visit MC.

7.5.2 Equivalence of Forward and Backward Views (Offline Updating)

Consider *offline* updating: increments are accumulated during the episode but only applied to V after the episode ends (so V used inside every δ_t is the fixed value function from the *start* of the episode). Let a state s be visited at time k in an episode of length T .

Theorem 7.5.1: Backward = Forward, exact identity

With $V(S_T) = 0$ (terminal states have value zero) and offline updating,

$$\sum_{t=k}^{T-1} (\gamma\lambda)^{t-k} \delta_t = G_k^\lambda - V(S_k).$$

Proof. We match coefficients of every term on both sides, treating both sides as linear combinations of the rewards $R_{k+1}, \dots, R_T$ and the (fixed, start-of-episode) values $V(S_{k+1}), \dots, V(S_{T-1})$.

Coefficient of R_{k+i+1} , for $0 \leq i \leq T - k - 1$. On the left, δ_t contains R_{t+1} with coefficient 1, so R_{k+i+1} appears only in the $t = k + i$ term, contributing $(\gamma\lambda)^i$. On the right, recall $G_k^{(n)} = \sum_{j=0}^{n-1} \gamma^j R_{k+j+1} + \gamma^n V(S_{k+n})$ contains R_{k+i+1} (coefficient γ^i) iff $n \geq i + 1$. Using the (standard, finite-episode) convention that the λ -return puts all residual weight λ^{T-k-1} on the final, terminal-bootstrap-free return $G_k^{(T-k)} = G_k$,

$$G_k^\lambda = (1 - \lambda) \sum_{n=1}^{T-k-1} \lambda^{n-1} G_k^{(n)} + \lambda^{T-k-1} G_k,$$

the coefficient of R_{k+i+1} is

$$\gamma^i \left[(1 - \lambda) \sum_{n=i+1}^{T-k-1} \lambda^{n-1} + \lambda^{T-k-1} \right] = \gamma^i \left[\lambda^i - \lambda^{T-k-1} + \lambda^{T-k-1} \right] = (\gamma\lambda)^i,$$

using the finite geometric sum $\sum_{n=i+1}^{T-k-1} \lambda^{n-1} = \lambda^i \frac{1 - \lambda^{T-k-1-i}}{1 - \lambda}$. This matches the left side exactly.

Coefficient of $V(S_j)$, for $k < j < T$. Write $j = k + n$. On the left, $V(S_j)$ appears in δ_{j-1} (as $+\gamma V(S_j)$) and in δ_j (as $-V(S_j)$), contributing

$$(\gamma\lambda)^{n-1} \gamma - (\gamma\lambda)^n = \gamma^n \lambda^{n-1} (1 - \lambda).$$

On the right, $V(S_j) = V(S_{k+n})$ appears only inside $G_k^{(n)}$, with coefficient γ^n , so its coefficient in G_k^λ is $(1 - \lambda) \lambda^{n-1} \gamma^n$ — an exact match.

Boundary term. At $t = k$, δ_k contributes $-V(S_k)$ with coefficient $(\gamma\lambda)^0 = 1$, matching the explicit $-V(S_k)$ on the right; at $t = T - 1$, the term $+\gamma V(S_T) = 0$ contributes nothing, matching the fact that G_k^λ contains no $V(S_T)$ term (the final n -step return $G_k^{(T-k)} = G_k$ has no bootstrap term).

Since every coefficient matches, the two sides are identical as linear functions of the observed rewards and values. ■

Corollary 7.5.1: TD(λ) = MC when $\lambda = 1$

Setting $\lambda = 1$ in Theorem 7.5.1 gives $\sum_{t=k}^{T-1} \gamma^{t-k} \delta_t = G_k - V(S_k)$: the total offline backward-view update at s (visited at time k) exactly equals the every-visit MC error. Summing over all visits within an episode, and over episodes, the total offline TD(1) update to V is identical to every-visit MC.

Corollary 7.5.2: Offline equivalence of forward and backward TD(λ)

Summing Theorem 7.5.1 over every time t at which state s is visited (and over every state s), the total offline update produced by the backward-view (eligibility-trace) algorithm at s equals the total offline update produced by the forward-view algorithm, $\alpha \sum_{k: S_k=s} (G_k^\lambda - V(S_k))$. Hence, for offline updating, forward-view and backward-view TD(λ) are exactly equivalent, for every $\lambda \in [0, 1]$.

Remark. For online updating (where V — and hence every subsequent δ_t — changes mid-episode), forward and backward view TD(λ) are only approximately equivalent; exact online TD(λ) (Sutton and van Seijen, ICML 2014) modifies the eligibility trace update slightly (the “dutch trace”) to restore exact equivalence step-by-step, not just in the offline total.

Exam Tip 7.5.1: What to have ready

- Reproduce Definition 7.1.1 and explain, in one line, why the TD target is biased but low-variance while the MC target is unbiased but high-variance.
- Reproduce the AB example (Example 7.2.1) numerically — it is a favourite because it cleanly separates MC and TD’s asymptotic answers on a tiny, hand-checkable dataset.
- Be able to state (and, time permitting, reproduce the coefficient-matching argument for) Theorem 7.5.1 — this is the “required derivation” most likely to appear verbatim on an exam given how central eligibility traces are to this chapter.

Chapter 8

Model-Free Control I: On-Policy Monte-Carlo Control

Status update. To solve a known MDP: Dynamic Programming (Chapter 5). To estimate the value function of an unknown MDP: model-free prediction (Chapters 6–7). Now: to *optimise* the value function of an *unknown* MDP, model-free control. Uses of model-free control range from elevator scheduling, Robocup soccer and parallel parking to portfolio management, protein folding and the game of Go — problems where the MDP is either unknown, or known but far too big to use except by sampling.

8.1 On- and Off-Policy Learning

Definition 8.1.1: On-Policy vs. Off-Policy Learning

- **On-policy learning:** “learn on the job.” Learn about policy π from experience *sampled from* π itself.
- **Off-policy learning:** “look over someone’s shoulder.” Learn about a target policy π from experience sampled from a *different* behaviour policy μ .

This chapter covers on-policy control via Monte-Carlo; on-policy TD control (Sarsa) is Chapter 9; off-policy control (Q-learning) is Chapter 10.

8.2 Generalised Policy Iteration with Monte-Carlo Evaluation

Naively we could plug MC policy evaluation (Chapter 6) directly into the GPI loop of Section 5.3: evaluate π with MC to get v_π , then improve greedily. This has two serious problems.

Key Idea 8.2.1: Problem 1: greedy improvement needs a model

Greedy policy improvement over $V(s)$ requires a model of the MDP:

$$\pi'(s) = \arg \max_{a \in \mathcal{A}} \left(R_s^a + \gamma \sum_{s'} P_{ss'}^a V(s') \right),$$

which needs P, R — exactly what we assumed is *unknown* in this part of the course. Solution: use the **action-value function** $Q(s, a)$ instead of $V(s)$. Greedy improvement over Q is

$$\pi'(s) = \arg \max_{a \in \mathcal{A}} Q(s, a),$$

which is completely model-free — exactly the point of q_π vs. v_π that Chapter 4 set up. This single substitution, $V \rightarrow Q$, is what makes *every* algorithm in this and the next two chapters model-free.

Key Idea 8.2.2: Problem 2: greedy exploitation can never explore

Even with Q , choosing $\pi' = \text{greedy}(Q)$ deterministically means some actions are *never* tried; if an early, lucky (or unlucky) sample gives a misleadingly high (or low) $Q(s, a)$ estimate, the greedy policy might lock onto a suboptimal action forever, since it never revisits the alternatives to correct the estimate. E.g. with two doors, one concealing a better long-run reward than the other: trying door A once and getting a mediocre payoff, followed by greedily always choosing whichever door currently looks better, can permanently miss a superior door B if B 's first sampled payoff happened to look worse. Solution: don't act *purely* greedily — use ε -greedy exploration.

Definition 8.2.1: ε -Greedy Exploration

The simplest idea for ensuring continual exploration: with probability $1 - \varepsilon$, choose the greedy action; with probability ε , choose an action uniformly at random (including, possibly, the greedy one again):

$$\pi(a | s) = \begin{cases} \varepsilon/|\mathcal{A}| + (1 - \varepsilon) & \text{if } a = \arg \max_{a' \in \mathcal{A}} Q(s, a') \\ \varepsilon/|\mathcal{A}| & \text{otherwise.} \end{cases}$$

Theorem 8.2.1: ε -Greedy Policy Improvement

For any ε -greedy policy π , the ε -greedy policy π' with respect to q_π is an improvement: $v_{\pi'}(s) \geq v_\pi(s)$ for all s .

Proof.

$$q_\pi(s, \pi'(s)) = \sum_{a \in \mathcal{A}} \pi'(a | s) q_\pi(s, a) = \frac{\varepsilon}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} q_\pi(s, a) + (1 - \varepsilon) \max_{a \in \mathcal{A}} q_\pi(s, a).$$

The max over a is at least as large as any weighted average of the same quantities, in particular the average with weights $\frac{\pi(a|s) - \varepsilon/|\mathcal{A}|}{1 - \varepsilon}$ (which are non-negative and sum to 1 because π is itself ε -greedy):

$$\max_{a \in \mathcal{A}} q_\pi(s, a) \geq \sum_{a \in \mathcal{A}} \frac{\pi(a | s) - \varepsilon/|\mathcal{A}|}{1 - \varepsilon} q_\pi(s, a).$$

Substituting this bound in,

$$q_\pi(s, \pi'(s)) \geq \frac{\varepsilon}{|\mathcal{A}|} \sum_a q_\pi(s, a) + \sum_a \left(\pi(a | s) - \frac{\varepsilon}{|\mathcal{A}|} \right) q_\pi(s, a) = \sum_{a \in \mathcal{A}} \pi(a | s) q_\pi(s, a) = v_\pi(s),$$

using Theorem 4.3.1 in the last step. So $q_\pi(s, \pi'(s)) \geq v_\pi(s)$ for all s , and the telescoping argument of Theorem 5.3.1 (which used only this one-step inequality, not that π be deterministic) applies verbatim to give $v_{\pi'}(s) \geq v_\pi(s)$. ■

8.2.1 Monte-Carlo Policy Iteration and Monte-Carlo Control

- **Policy evaluation:** Monte-Carlo policy evaluation, $Q \approx q_\pi$.
- **Policy improvement:** ε -greedy policy improvement.

Monte-Carlo Control tightens this loop to update *every episode* rather than waiting for full convergence of Q (another instance of Generalised Policy Iteration, Definition 5.3.1): after each episode, update Q by incremental MC (Definition 6.2.1, applied to $Q(S_t, A_t)$ instead of $V(S_t)$), then improve $\pi \leftarrow \varepsilon$ -greedy(Q).

8.3 GLIE and Convergence to the Optimal Policy

ε -greedy exploration with a *fixed* $\varepsilon > 0$ ensures continual exploration but also means the policy never becomes fully greedy, so it cannot converge exactly to π_* (it keeps taking a fraction ε of random, possibly-suboptimal actions forever). We need ε to vanish, but not too fast.

Definition 8.3.1: GLIE: Greedy in the Limit with Infinite Exploration

A sequence of policies $\pi_1, \pi_2, \dots$ is GLIE if:

- (i) all state-action pairs are explored infinitely many times,

$$\lim_{k \rightarrow \infty} N_k(s, a) = \infty \quad \forall s, a;$$

- (ii) the policy converges on a greedy policy,

$$\lim_{k \rightarrow \infty} \pi_k(a | s) = \mathbf{1}(a = \arg \max_{a'} Q_k(s, a')).$$

Worked Example 8.3.1: A GLIE schedule

ε -greedy is GLIE if ε is reduced to zero at $\varepsilon_k = 1/k$ (with k the episode index): $\sum_k \varepsilon_k = \infty$ ensures every action keeps getting sampled (satisfying condition (i)), while $\varepsilon_k \rightarrow 0$ ensures the policy becomes greedy in the limit (condition (ii)).

Algorithm 3: GLIE Monte-Carlo Control

```

1 for each episode  $k$  do
2   Sample an episode using  $\pi$ :  $\{S_1, A_1, R_2, \dots, S_T\} \sim \pi$ ;
3   for each state  $S_t$  and action  $A_t$  in the episode do
4      $N(S_t, A_t) \leftarrow N(S_t, A_t) + 1$ ;
5      $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{1}{N(S_t, A_t)}(G_t - Q(S_t, A_t))$ ;
6   Set  $\varepsilon \leftarrow 1/k$ ;
7    $\pi \leftarrow \varepsilon$ -greedy( $Q$ );
```

Theorem 8.3.1: Convergence of GLIE Monte-Carlo Control

GLIE Monte-Carlo control converges to the optimal action-value function, $Q(s, a) \rightarrow q_*(s, a)$, and hence the derived policy converges to an optimal policy π_* .

Remark. *Intuitively: condition (i) of GLIE guarantees every $Q(s, a)$ keeps being refined by the law of large numbers (as in ordinary MC policy evaluation, Chapter 6); condition (ii) guarantees the policy used to generate experience converges to a greedy (and hence eventually near-optimal) one, so that in the limit we are evaluating (and simultaneously acting according to) the greedy policy with respect to a Q that itself converges — pinning down the unique fixed point q_* via the same GPI argument as Theorem 5.3.2.*

Worked Example 8.3.2: Monte-Carlo Control in Blackjack

Applying GLIE MC control to the Blackjack MDP of Example 6.1.1 (now learning $Q(s, a)$ for $a \in \{\text{stick}, \text{twist}\}$, with no model of the dealer's behaviour ever specified) converges, after a few hundred thousand simulated hands, to the textbook-optimal Blackjack strategy: broadly, twist on low sums, stick on high sums, with the exact threshold shifted by whether a usable ace is held and by the dealer's visible card.

Exam Tip 8.3.1: The two fixes to remember

If asked “why can we not just do policy iteration with Monte-Carlo policy evaluation directly, model-free?” the expected two-part answer is exactly Key Idea 8.2.1 (use Q , not V) and Key Idea 8.2.2 (use ϵ -greedy, not greedy) — and if asked for convergence guarantees, the GLIE conditions (Definition 8.3.1) are the precise statement of “explore enough, but eventually stop exploring.”

Chapter 9

Model-Free Control II: Sarsa and Sarsa(λ)

Chapter 8 plugged Monte-Carlo evaluation into the GPI loop for control. This chapter makes the analogous substitution with Temporal-Difference evaluation, inheriting all of TD's advantages over MC (Chapter 7): lower variance, online learning, and applicability to non-terminating environments.

9.1 From MC Control to TD Control

Natural idea: apply TD instead of MC to the control loop.

- Apply TD to $Q(S, A)$ instead of $V(S)$.
- Use ε -greedy policy improvement (Definition 8.2.1).
- Update *every time-step* rather than waiting for the end of an episode (unlocking online learning and non-episodic tasks, exactly TD's selling point in Chapter 7).

9.2 Sarsa

Definition 9.2.1: Sarsa Update

Given a transition S, A, R, S', A' (hence the name: **State, Action, Reward, State, Action**), update

$$Q(S, A) \leftarrow Q(S, A) + \alpha(R + \gamma Q(S', A') - Q(S, A)).$$

This is exactly the TD(0) update of Definition 7.1.1 with V replaced by Q and the successor state S_{t+1} replaced by the successor *state-action pair* (S_{t+1}, A_{t+1}) — the same substitution that made MC control model-free now makes TD control model-free, and the action A' actually taken next (sampled from the *same* ε -greedy policy currently being followed) is what makes Sarsa **on-policy**: the target uses the real next action the agent is actually about to take, not a hypothetical best action.

Algorithm 4: On-Policy Control with Sarsa

```
1 Initialise  $Q(s, a)$  arbitrarily (e.g.  $Q(\text{terminal}, \cdot) = 0$ );
2 for each episode do
3   Initialise  $S$ ; choose  $A$  from  $S$  using policy derived from  $Q$  (e.g.  $\varepsilon$ -greedy);
4   for each step of the episode do
5     Take action  $A$ , observe  $R, S'$ ;
6     Choose  $A'$  from  $S'$  using policy derived from  $Q$  (e.g.  $\varepsilon$ -greedy);
7      $Q(S, A) \leftarrow Q(S, A) + \alpha(R + \gamma Q(S', A') - Q(S, A))$ ;
8      $S \leftarrow S'; A \leftarrow A'$ ;
```

Theorem 9.2.1: Convergence of Sarsa

Sarsa converges to the optimal action-value function, $Q(s, a) \rightarrow q_*(s, a)$, under the following conditions:

- (i) a GLIE sequence of policies $\pi_t(a | s)$ (Definition 8.3.1);
- (ii) a Robbins–Monro sequence of step-sizes α_t :

$$\sum_{t=1}^{\infty} \alpha_t = \infty \quad \text{and} \quad \sum_{t=1}^{\infty} \alpha_t^2 < \infty.$$

Key Idea 9.2.1: Reading the Robbins–Monro conditions

$\sum_t \alpha_t = \infty$: the steps must be large enough, in total, that any starting point can eventually be corrected — Q can always move far enough to reach q_* no matter how far off the initialisation was. $\sum_t \alpha_t^2 < \infty$: the steps must shrink fast enough that accumulated noise does not prevent convergence — the variance of the cumulative update stays bounded. A constant $\alpha_t = \alpha$ violates the second condition (useful for tracking non-stationary targets, as noted in Definition 6.2.1, but it does not give exact convergence); $\alpha_t = 1/t$ satisfies both.

Worked Example 9.2.1: Windy Gridworld

A gridworld with a crosswind running upward through the middle columns (strength 0, 0, 0, 1, 1, 1, 2, 2, 1, 0 column-by-column), start cell S , goal cell G , standard 4-connected moves, reward -1 per time-step, undiscounted. Sarsa learns a policy that compensates for the wind's push, and (unlike a purely greedy or DP approach) can be run online, step by step, on a *continuing* task with no natural episode boundary other than reaching G — there is no way to apply Monte-Carlo control here as directly, since Sarsa starts learning and improving *within* the very first episode.

9.3 Sarsa(λ)

Exactly as n -step TD interpolated between TD(0) and MC for *prediction* (Chapter 7), we can interpolate Sarsa across multiple steps for *control*.

Definition 9.3.1: n -Step Q-Return

$$q_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n Q(S_{t+n}, A_{t+n}).$$

Definition 9.3.2: Forward-View Sarsa(λ)

$$q_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} q_t^{(n)}, \quad Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha (q_t^\lambda - Q(S_t, A_t)),$$

a direct copy of Definitions 7.4.2 and 7.4.3 with $V \rightarrow Q$ and states replaced by state-action pairs.

Definition 9.3.3: Backward-View Sarsa(λ)

Maintain one eligibility trace *per state-action pair*:

$$E_0(s, a) = 0, \quad E_t(s, a) = \gamma \lambda E_{t-1}(s, a) + \mathbf{1}(S_t = s, A_t = a),$$

and at every step update every (s, a) in proportion to its trace, using the Sarsa TD error $\delta_t = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)$:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \delta_t E_t(s, a) \quad \forall s, a.$$

By the same offline-equivalence argument as Theorem 7.5.1 (applied coordinate-wise to (s, a) -pairs instead of states), the total offline update produced by the backward view exactly matches the forward view, for every λ .

Algorithm 5: Sarsa(λ)

```

1 Initialise  $Q(s, a)$  arbitrarily, for all  $s, a$ ;
2 for each episode do
3    $E(s, a) \leftarrow 0$  for all  $s, a$ ;
4   Initialise  $S, A$ ;
5   for each step of the episode do
6     Take action  $A$ , observe  $R, S'$ ;
7     Choose  $A'$  from  $S'$  using policy derived from  $Q$  (e.g.  $\epsilon$ -greedy);
8      $\delta \leftarrow R + \gamma Q(S', A') - Q(S, A)$ ;
9      $E(S, A) \leftarrow E(S, A) + 1$ ;
10    for all  $s, a$  do
11       $Q(s, a) \leftarrow Q(s, a) + \alpha \delta E(s, a)$ ;
12       $E(s, a) \leftarrow \gamma \lambda E(s, a)$ ;
13     $S \leftarrow S'; A \leftarrow A'$ ;

```

Worked Example 9.3.1: Sarsa(λ) on the Gridworld

Consider a single episode that reaches the goal after many steps, with reward only at the goal. Plain Sarsa (i.e. $\lambda = 0$) only updates the *single* state-action pair immediately preceding the goal after that episode — every earlier step on the same path must wait for future episodes to have its value propagated backward, one step per episode. Sarsa(λ) with $\lambda > 0$ instead updates *every* (s, a) pair visited earlier in that same episode immediately, in one sweep, weighted by its eligibility trace — so useful information from a single successful episode propagates all the way back to the start state at once, dramatically speeding up learning compared to $\lambda = 0$, without the full variance cost of waiting for many-step or full-episode (MC-like) returns.

Exam Tip 9.3.1: Sarsa vs. Sarsa(λ) vs. MC control

A good way to organise this chapter for revision: Sarsa is to TD(0) what Monte-Carlo control (Chapter 8) is to MC prediction, and Sarsa(λ) is to TD(λ) (Chapter 7) what Sarsa is to TD(0). Every derivation in this chapter is literally a $V \rightarrow Q$, state $\rightarrow$ state-action substitution into a chapter you have already seen — if you understand Chapter 7 cold, this chapter costs almost nothing extra.

Chapter 10

Off-Policy Learning and Q-Learning

Every algorithm so far has been **on-policy**: we evaluate and improve the very policy used to generate experience. This chapter asks: can we evaluate (or optimise) a *target* policy π while acting according to a *different behaviour* policy μ ? The answer leads first to importance sampling, and then — much more elegantly — to Q-learning, the single most influential algorithm in this course.

10.1 Why Off-Policy Learning?

Motivations for learning about π while following μ :

- Learn from observing humans or other (non-learning) agents.
- Re-use experience generated from *old* policies $\pi_1, \pi_2, \dots, \pi_{t-1}$ — valuable because generating new experience is expensive, and old trajectories should not simply be thrown away.
- Learn about an *optimal* policy while following an *exploratory* policy — resolving the exploration/exploitation dilemma in a new way: keep exploring with μ , but still learn about the greedy π .
- Learn about *multiple* policies while following *one* — e.g. learning many different "what if" sub-goals from a single stream of experience.

10.2 Importance Sampling

Proposition 10.2.1: Importance Sampling Identity

For any function f and distributions P, Q with $Q(x) > 0$ whenever $P(x)f(x) \neq 0$,

$$\mathbb{E}_{X \sim P}[f(X)] = \sum_x P(x)f(x) = \sum_x Q(x) \frac{P(x)}{Q(x)} f(x) = \mathbb{E}_{X \sim Q} \left[\frac{P(X)}{Q(X)} f(X) \right].$$

Proof. Multiply and divide the summand by $Q(x)$, then read the result as an expectation under Q . ■

This lets us estimate an expectation under one distribution using samples drawn from another, reweighting each sample by the likelihood ratio (the *importance-sampling ratio* or *importance weight*).

10.2.1 Importance Sampling for Off-Policy Monte-Carlo

Use returns generated from μ to evaluate π : weight the return G_t by the ratio of the two policies' probabilities along the *entire* trajectory, correcting for every action taken along the way:

$$G_t^{\pi/\mu} = \frac{\pi(A_t | S_t)}{\mu(A_t | S_t)} \cdot \frac{\pi(A_{t+1} | S_{t+1})}{\mu(A_{t+1} | S_{t+1})} \cdots \frac{\pi(A_T | S_T)}{\mu(A_T | S_T)} G_t,$$

and update $V(S_t) \leftarrow V(S_t) + \alpha(G_t^{\pi/\mu} - V(S_t))$.

Key Idea 10.2.1: Why this is rarely used in practice

Each ratio $\pi(A_k | S_k)/\mu(A_k | S_k)$ is itself a random variable; their *product* over a whole episode has variance that compounds multiplicatively over every step, and can (if μ ever assigns near-zero probability to an action π favours) blow up without bound. Worse, if μ is ever zero where π is not, importance sampling is simply undefined. Combined with MC's already-high variance (Proposition 7.1.2), this makes off-policy MC importance sampling impractically noisy.

10.2.2 Importance Sampling for Off-Policy TD

Use TD targets generated from μ to evaluate π , weighting by a *single-step* probability ratio (not a whole-episode product):

$$V(S_t) \leftarrow V(S_t) + \alpha \left(\frac{\pi(A_t | S_t)}{\mu(A_t | S_t)} (R_{t+1} + \gamma V(S_{t+1})) - V(S_t) \right).$$

Much lower variance than the Monte-Carlo version, since only one ratio is involved instead of a whole-episode product; policies only need to agree over a single step, not over a whole trajectory. Still, we can do better still — for the specific case of *control*, the next section shows importance sampling can be avoided *entirely*.

10.3 Q-Learning

Now consider off-policy learning of *action-values* $Q(s, a)$, with no importance sampling required.

Key Idea 10.3.1: The key trick

Choose the *next* action actually taken, A_{t+1} , from the behaviour policy $\mu(\cdot | S_t)$ (e.g. ϵ -greedy, to keep exploring). But choose the *alternative*, hypothetical next action A' used to *construct the update target* from the **target policy** $\pi(\cdot | S_t)$ instead. Then update $Q(S_t, A_t)$ toward the value of that alternative action:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma Q(S_{t+1}, A') - Q(S_t, A_t)).$$

Because A_t itself was sampled from μ (not π) but the reward and next-state transition R_{t+1}, S_{t+1} depend only on (S_t, A_t) — not on which *policy* chose A_t — and the only place π enters is inside the target, choosing $A' \sim \pi$ requires *no* importance-sampling correction at all: we are simply asking “what would the target policy have done here, and how good is that?”, which is a well-defined, unbiased question regardless of how S_t or A_t came to be observed.

10.3.1 Off-Policy Control with Q-Learning

Now let *both* policies improve:

- The **target** policy π is taken to be *greedy* w.r.t. $Q(s, a)$: $\pi(S_{t+1}) = \arg \max_{a'} Q(S_{t+1}, a')$.
- The **behaviour** policy μ is taken to be ε -*greedy* w.r.t. the very same $Q(s, a)$ (so the agent still explores while acting).

Proposition 10.3.1: Simplification of the Q-Learning Target

With π exactly greedy, the target simplifies:

$$R_{t+1} + \gamma Q(S_{t+1}, \arg \max_{a'} Q(S_{t+1}, a')) = R_{t+1} + \gamma \max_{a' \in \mathcal{A}} Q(S_{t+1}, a').$$

Definition 10.3.1: Q-Learning Update

$$Q(S, A) \leftarrow Q(S, A) + \alpha \left(R + \gamma \max_{a' \in \mathcal{A}} Q(S', a') - Q(S, A) \right).$$

Key Idea 10.3.2: Reading the update: this is a sampled Bellman *Optimality* backup

Compare with the Bellman Optimality Equation for q_* (Theorem 4.4.5): $q_*(s, a) = R_s^a + \gamma \sum_{s'} P_{ss'}^a \max_{a'} q_*(s', a')$. Q-learning's target $R + \gamma \max_{a'} Q(S', a')$ is exactly a single-sample, bootstrapped version of the right-hand side — the same relationship TD(0) had to the (expectation) Bellman equation, but now for the *optimality* equation directly. This is why Q-learning is off-policy *by its very structure*: its target is always the value of acting *optimally* from S' onward, regardless of what policy was actually used to reach S' or will be used to choose the next real action.

Theorem 10.3.1: Convergence of Q-Learning

Q-learning control converges to the optimal action-value function, $Q(s, a) \rightarrow q_*(s, a)$, under conditions closely analogous to Sarsa's (Theorem 9.2.1): every state-action pair continues to be visited infinitely often (ensured by, e.g., an ε -greedy behaviour policy with ε not decayed to 0 too fast, or any policy that never stops exploring), and a Robbins–Monro step-size schedule. Unlike Sarsa, convergence does *not* require the behaviour policy itself to become greedy in the limit (no GLIE requirement on μ) — Q converges to q_* even if μ keeps exploring forever, precisely because the target policy embedded in the update is already exactly greedy at every step.

Algorithm 6: Q-Learning for Off-Policy Control

```

1 Initialise  $Q(s, a)$  arbitrarily (e.g.  $Q(\text{terminal}, \cdot) = 0$ );
2 for each episode do
3   Initialise  $S$ ;
4   for each step of the episode do
5     Choose  $A$  from  $S$  using policy derived from  $Q$  (e.g.  $\varepsilon$ -greedy);
6     Take action  $A$ , observe  $R, S'$ ;
7      $Q(S, A) \leftarrow Q(S, A) + \alpha \left( R + \gamma \max_{a'} Q(S', a') - Q(S, A) \right)$ ;
8      $S \leftarrow S'$ ;
```

Worked Example 10.3.1: Cliff Walking: Sarsa vs. Q-Learning

A gridworld with a start S and goal G along the bottom row, and a "cliff" occupying the rest of the bottom row between them: stepping into the cliff gives reward -100 and resets to S ; every other step costs -1 . There are two qualitatively different paths: a *safe* path

along the top (long, but zero risk) and an *optimal* (shortest) path running directly along the cliff edge.

- **Q-learning** learns the value of the *optimal* greedy path right next to the cliff, since its target always bootstraps off $\max_{a'} Q(S', a')$, i.e. the value of behaving optimally from S' onward — irrespective of the fact that its own ε -greedy behaviour will occasionally still step off the cliff while learning.
- **Sarsa** learns the value of the safer path further from the cliff, because its target $Q(S', A')$ uses the action A' actually about to be taken by the (still ε -greedy, still occasionally-random) policy currently in force — so it correctly accounts for the real, nonzero probability of an exploratory step sending the agent over the edge near the optimal path, and prices that risk into the learned values.

Consequently, Q-learning’s optimal-path policy achieves a *better* asymptotic (fully-greedy, $\varepsilon \rightarrow 0$) return, but a *worse* online, “sum of rewards actually received while still exploring with $\varepsilon > 0$ ” return than Sarsa, which is the more common way this example’s learning curves are presented: Sarsa’s online reward-per-episode curve sits visibly above Q-learning’s throughout training, illustrating on-policy vs. off-policy as a genuinely practical, not just theoretical, distinction — on-policy methods optimise the policy you actually run; off-policy methods optimise the policy you say you *want*, even while running something else.

10.4 The Relationship Between DP and TD

We can now complete the unifying picture first sketched with the backup diagrams of Figure 7.1: every full-width (DP) algorithm has a sample-based (TD) analogue using the *same* underlying Bellman equation.

	Full Backup (DP)	Sample Backup (TD)
Bellman Expectation Equation for $v_\pi(s)$	Iterative Policy Evaluation (Section 5.2)	TD Learning (Chapter 7)
Bellman Expectation Equation for $q_\pi(s, a)$	Q-Policy Iteration (DP analogue of Section 5.3 using q_π)	Sarsa (Chapter 9)
Bellman Optimality Equation for $q_*(s, a)$	Q-Value Iteration (DP analogue of Section 5.4 using q_*)	Q-Learning (Section 10.3)

Table 10.1: The relationship between DP and TD. Every row is the same Bellman equation; the left column averages over successors exactly (needs a model); the right column samples one successor and bootstraps (needs no model). This table is one of the most re-drawn diagrams in all of RL, and a very efficient way to hold the entire course in your head at once.

Exam Tip 10.4.1: The single most important insight in this chapter

If you remember one thing from Chapters 8–10, make it this: Q-learning needs *no* importance sampling, precisely *because* its target already bakes in a fixed, deterministic (greedy) target policy via the max operator, rather than averaging over a stochastic target policy the way Section 5.2-style prediction does. Contrast this crisply with off-policy TD *prediction* (evaluating some fixed stochastic $\pi \neq \mu$), which genuinely does need the importance-sampling ratio $\pi(A_t|S_t)/\mu(A_t|S_t)$ — make sure you can explain, in one sentence, why control-via-max

sidesteps exactly the problem that prediction-via-averaging cannot.

Chapter 11

Value Function Approximation

Every algorithm so far stored V or Q as an explicit *table*, one entry per state (or state-action pair). This does not scale: backgammon has $\sim 10^{20}$ states, computer Go $\sim 10^{170}$, a helicopter's state space is continuous, and protein folding has 20^N configurations for chain length N up to 2500. This chapter replaces the table with a parametric, *generalising* function approximator, so that updating the value at one state usefully changes the estimated value at many similar states.

11.1 Value Function Approximation

Definition 11.1.1: Value Function Approximation

Represent the value function by a parametric function of a weight vector $w \in \mathbb{R}^n$:

$$\hat{v}(s, w) \approx v_\pi(s), \quad \text{or} \quad \hat{q}(s, a, w) \approx q_\pi(s, a),$$

with $n \ll |\mathcal{S}|$ (or $|\mathcal{S} \times \mathcal{A}|$). Updating w from one piece of experience generalises to *many* states/actions at once, and the function can represent value functions for continuous state or action spaces.

Types of approximator: linear combinations of features, neural networks, decision trees, nearest neighbour, Fourier/wavelet bases, and more. This course restricts attention to **differentiable** function approximators (linear combinations of features and neural networks being the two dominant examples), since we will fit them by gradient descent, using a training method suited to **non-stationary, non-i.i.d.** data — successive states visited by an RL agent are correlated and the target itself keeps changing as w (and, in control, π) changes, unlike the i.i.d. assumption of ordinary supervised learning.

11.2 Gradient Descent

For a differentiable function $J(w)$, define the gradient $\nabla_w J(w) = (\partial J / \partial w_1, \dots, \partial J / \partial w_n)^\top$. Gradient descent adjusts w downhill:

$$\Delta w = -\frac{1}{2} \alpha \nabla_w J(w),$$

with α a step-size parameter (the $\frac{1}{2}$ is a convenience that cancels a factor of 2 from differentiating a square, as below).

11.2.1 Value function approximation by SGD

Definition 11.2.1: Objective for Prediction

Goal: find w minimising the mean-squared error between the true value function $v_\pi(S)$ and its approximation $\hat{v}(S, w)$, over the distribution of states actually visited under π :

$$J(w) = \mathbb{E}_\pi \left[(v_\pi(S) - \hat{v}(S, w))^2 \right].$$

Proposition 11.2.1: Gradient of the Objective

$$\Delta w = -\frac{1}{2}\alpha \nabla_w J(w) = \alpha \mathbb{E}_\pi [(v_\pi(S) - \hat{v}(S, w)) \nabla_w \hat{v}(S, w)].$$

Proof. $\nabla_w (v_\pi(S) - \hat{v}(S, w))^2 = 2(v_\pi(S) - \hat{v}(S, w)) \cdot (-\nabla_w \hat{v}(S, w))$ by the chain rule (matching the second matrix-calculus identity of Chapter 1); take $\mathbb{E}_\pi[\cdot]$, multiply by $-\frac{1}{2}\alpha$. ■

Stochastic gradient descent samples a single state S (rather than computing the full expectation) and takes a step toward it:

$$\Delta w = \alpha (v_\pi(S) - \hat{v}(S, w)) \nabla_w \hat{v}(S, w).$$

Even though each individual sample is noisy, the expected update equals full gradient descent, so SGD converges (under standard step-size conditions) to a local (for non-linear $\hat{v}$) or global (for linear $\hat{v}$, next section) optimum of $J(w)$.

11.3 Linear Value Function Approximation

Definition 11.3.1: Feature Vectors

Represent each state by a feature vector $x(S) = (x_1(S), \dots, x_n(S))^\top$, e.g. distance from landmarks (robot), trend indicators (stock market), piece/pawn configurations (chess).

Definition 11.3.2: Linear Value Function Approximation

$$\hat{v}(S, w) = x(S)^\top w = \sum_{j=1}^n x_j(S) w_j.$$

Proposition 11.3.1: Linear SGD update

For linear $\hat{v}$, $\nabla_w \hat{v}(S, w) = x(S)$ (Chapter 1), so the SGD update collapses to a strikingly simple rule:

$$\Delta w = \alpha (v_\pi(S) - \hat{v}(S, w)) x(S).$$

Key Idea 11.3.1: Why linear is special: a unique global optimum

$J(w)$ is a *quadratic* function of w for the linear case (it is a sum of squares of an affine function of w), so it has no local optima other than the global one — SGD (indeed, even ordinary gradient descent) provably converges to the globally best w (subject to step-size conditions), never getting stuck. This single fact is why linear VFA has by far the strongest theoretical guarantees in this chapter, and it is also why Section 11.7 can solve for the optimal w *directly*, in closed form, rather than only iteratively.

Remark. *Table lookup* is the special case of linear VFA where $x(S)$ is a one-hot indicator vector, $x(S) = (\mathbf{1}(S = s_1), \dots, \mathbf{1}(S = s_{|S|}))^\top$, and w_i literally is $V(s_i)$: every algorithm in

Chapters 5–10 is thus a special case of the algorithms in this chapter, with $n = |S|$ and no generalisation across states at all.

11.4 Incremental Prediction Algorithms

There is one obstacle: $v_\pi(S)$ (the “true” target in Definition 11.2.1) is exactly what we don’t know in model-free RL. Following the same pattern as Chapters 6 and 7, we substitute a target we *can* compute for $v_\pi(S)$.

- **Monte-Carlo** with VFA: target = G_t , giving $\Delta w = \alpha(G_t - \hat{v}(S_t, w)) \nabla_w \hat{v}(S_t, w)$. Since G_t is (as established in Proposition 7.1.1) an unbiased, if noisy, sample of $v_\pi(S_t)$, this is literally supervised learning on noisy labels (S_t, G_t) pairs, and inherits SGD’s usual guarantees: converges to a local optimum (a global optimum for linear VFA), even for non-linear function approximators.
- **TD(0)** with VFA: target = $R_{t+1} + \gamma \hat{v}(S_{t+1}, w)$, giving

$$\Delta w = \alpha(R_{t+1} + \gamma \hat{v}(S_{t+1}, w) - \hat{v}(S_t, w)) \nabla_w \hat{v}(S_t, w).$$

This target depends on the current weights w *itself* (via $\hat{v}(S_{t+1}, w)$), which is what makes it a *biased* sample of $v_\pi(S_t)$ (as in Proposition 7.1.1) and technically means this is only a “semi-gradient” method — we differentiate only through $\hat{v}(S_t, w)$, treating the bootstrapped target as fixed, ignoring the w -dependence hidden inside it. Despite not being a true gradient of any single objective, linear TD(0) is proven to converge (close) to the global optimum; convergence with genuinely non-linear function approximation is not guaranteed in general (see Section 11.6).

- **TD(λ)** with VFA: forward view uses target G_t^λ (the λ -return of Definition 7.4.2, now built from $\hat{v}(\cdot, w)$ bootstraps); backward view keeps one eligibility trace *per weight component* rather than per state:

$$E_t = \gamma \lambda E_{t-1} + \nabla_w \hat{v}(S_t, w), \quad \Delta w = \alpha \delta_t E_t,$$

with $\delta_t = R_{t+1} + \gamma \hat{v}(S_{t+1}, w) - \hat{v}(S_t, w)$. For linear $\hat{v}$, $\nabla_w \hat{v}(S_t, w) = x(S_t)$, so this reduces to the familiar per-feature trace $E_t = \gamma \lambda E_{t-1} + x(S_t)$; forward-view and backward-view linear TD(λ) remain (offline-)equivalent, by the same telescoping argument as Theorem 7.5.1 applied component-wise to each weight.

11.5 Control with Value Function Approximation

Follow Generalised Policy Iteration (Definition 5.3.1) once more, now with approximate policy evaluation: $\hat{q}(\cdot, \cdot, w) \approx q_\pi$, and ε -greedy improvement over $\hat{q}(\cdot, \cdot, w)$.

Definition 11.5.1: Linear Action-Value Function Approximation

Feature vector over state-action pairs $x(S, A)$, and $\hat{q}(S, A, w) = x(S, A)^\top w$, with SGD update $\Delta w = \alpha(q_\pi(S, A) - \hat{q}(S, A, w)) \nabla_w \hat{q}(S, A, w) = \alpha(q_\pi(S, A) - \hat{q}(S, A, w)) x(S, A)$.

Substituting a computable target for $q_\pi(S, A)$, exactly as before, gives the incremental control algorithms:

- **MC control with VFA**: target = G_t .
- **Sarsa with VFA**: target = $R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, w)$.
- **Sarsa(λ) with VFA**: forward view uses the q - λ -return of Definition 9.3.2 built from $\hat{q}(\cdot, \cdot, w)$; backward view keeps a trace $E_t = \gamma \lambda E_{t-1} + \nabla_w \hat{q}(S_t, A_t, w)$ and updates $\Delta w = \alpha \delta_t E_t$.

Worked Example 11.5.1: Mountain Car with Linear Sarsa

A car on a 1-D track between two hills must reverse up the far hill to build momentum before it can reach the goal at the top of the near hill, since its engine is too weak to climb directly. State = (position, velocity), continuous. Using **coarse coding** or **radial basis functions** (RBFs) — overlapping receptive-field features, each active over a region of the continuous $(x, \dot{x})$ plane, giving $x(s)$ a sparse, localised, generalising representation — linear Sarsa(λ) learns a smooth “cost-to-go” surface over the continuous state space directly, with no discretisation into an explicit table, and empirically performs best for some intermediate λ (mirroring Example 7.4.1), with *replacing* traces (cap each trace at 1 rather than letting repeated visits accumulate past 1) typically outperforming naive *accumulating* traces for this kind of task.

11.6 Convergence Issues: The Deadly Triad**Key Idea 11.6.1: The deadly triad**

Combining all three of (i) function approximation, (ii) bootstrapping, and (iii) off-policy training is dangerous: any two together are typically safe, but all three together can cause the weights w to **diverge to infinity**, even on simple problems with a bounded true value function. TD does not follow the gradient of *any* fixed objective function (recall the “semi-gradient” caveat above) — this is precisely why TD, unlike true gradient descent, can diverge when bootstrapping under off-policy sampling with (linear or non-linear) function approximation.

Worked Example 11.6.1: Baird’s Counterexample

A small MDP (one “central” state plus several “outer” states, all absorbing under the target policy, with a behaviour policy that visits the outer states far more often than the on-policy state distribution would) with a linear feature representation deliberately constructed so that the outer states’ weights are only weakly tied to the reward signal. Under off-policy (uniformly-random-behaviour, always-target-the-same-action) DP-style or TD(0) full/semi-gradient backups, the weight vector provably diverges to $\pm\infty$ even though $\gamma < 1$ and every reward is bounded — a stark, minimal demonstration that (ii)+(iii) together, combined with (i), break the contraction-mapping guarantees of Chapter 5: the off-policy state distribution used to weight the (now approximate) Bellman backup is no longer the one under which the backup operator is guaranteed to be a non-expansion.

On/Off-Policy	Algorithm	Table Lookup	Linear	Non-Linear
On-Policy	MC	✓	✓	✓
	TD(0)	✓	✓	×
	TD(λ)	✓	✓	×
Off-Policy	MC	✓	✓	✓
	TD(0)	✓	×	×
	TD(λ)	✓	×	×

Table 11.1: Convergence of prediction algorithms. MC always converges (it is true, unbiased gradient descent on $J(w)$); off-policy TD can diverge as soon as any function approximation (even linear) is introduced — Baird’s counterexample lives exactly in that cell.

Algorithm	Table Lookup	Linear	Non-Linear
Monte-Carlo Control	✓	(✓)	×
Sarsa	✓	(✓)	×
Q-Learning	✓	×	×
Gradient Q-Learning	✓	✓	×

Table 11.2: Convergence of control algorithms. “(✓)” = chatters around a near-optimal value function without settling exactly. Notice Q-learning — off-policy by construction (Section 10.3) — loses even the linear guarantee that on-policy Sarsa retains.

Key Idea 11.6.2: Gradient TD: fixing the divergence

Ordinary TD does not follow the gradient of any objective, which is the root cause of the divergence above. **Gradient TD** methods instead follow the true (stochastic) gradient of an explicit objective — the *projected Bellman error* — restoring convergence guarantees under linear function approximation even off-policy, at the cost of a more involved (two-timescale) update rule. This is a modern fix; conceptually, file it under “choose your target so that you are provably doing gradient descent on something.”

11.7 Batch Methods

Gradient descent, as above, is simple but **not sample-efficient**: each piece of experience is used for one small update and then discarded. Batch methods instead seek the best-fitting value function *given the agent’s entire stored experience* (its “training data”).

11.7.1 Least Squares Prediction and Experience Replay

Definition 11.7.1: Least Squares Objective

Given experience $D = \{(s_1, v_1^\pi), \dots, (s_T, v_T^\pi)\}$ (with v_t^π some target — G_t for the MC version, a TD target for the TD version), find w minimising the sum-squared error *on this fixed dataset*:

$$LS(w) = \sum_{t=1}^T (v_t^\pi - \hat{v}(s_t, w))^2.$$

Definition 11.7.2: Stochastic Gradient Descent with Experience Replay

Repeat: (i) sample a state, target pair (s, v^π) from stored experience D (uniformly, or otherwise); (ii) apply the ordinary SGD update $\Delta w = \alpha(v^\pi - \hat{v}(s, w))\nabla_w \hat{v}(s, w)$. In expectation over repeated sampling from D , this converges to the least-squares solution of Definition 11.7.1 for that fixed batch — reusing each transition many times instead of once. **Deep Q-Networks (DQN)** combine experience replay with **fixed Q-targets** (freezing the target network’s weights for many steps at a time, updating only the online network) to stabilise non-linear, off-policy Q-learning — directly attacking the deadly triad by decorrelating updates (replay) and slowing the moving-target problem (fixed targets).

11.7.2 Linear Least Squares Prediction: a closed-form solution

Experience replay still needs many iterations. For **linear** VFA we can instead solve for the least-squares w *directly*, exactly as Proposition 3.3.1 solved the tabular Bellman equation directly.

Proposition 11.7.1: Direct (Batch) Solution — LSMC and LSTD

At the least-squares optimum, the gradient of $LS(w)$ vanishes:

$$0 = \sum_{t=1}^T x(s_t)(v_t^\pi - x(s_t)^\top w) \implies w = \left(\sum_{t=1}^T x(s_t)x(s_t)^\top \right)^{-1} \sum_{t=1}^T x(s_t)v_t^\pi.$$

- **LSMC (Least-Squares Monte-Carlo)**: set $v_t^\pi = G_t$ directly in the formula above.
- **LSTD (Least-Squares TD)**: since the TD target $R_{t+1} + \gamma x(s_{t+1})^\top w$ itself contains w , solve instead for the fixed point of the *expected TD update* being zero,

$$0 = \sum_{t=1}^T x(s_t) \left(R_{t+1} + \gamma x(s_{t+1})^\top w - x(s_t)^\top w \right) \implies w = A^{-1}b,$$

$$\text{where } A = \sum_{t=1}^T x(s_t)(x(s_t) - \gamma x(s_{t+1}))^\top \text{ and } b = \sum_{t=1}^T x(s_t) R_{t+1}.$$

Remark. Direct solution costs $O(n^3)$ (matrix inverse in the feature dimension n , not the state count $|\mathcal{S}|$ — the whole point of using features with $n \ll |\mathcal{S}|$); an incremental $O(n^2)$ -per-step version exists via the Sherman–Morrison formula, updating A^{-1} directly without ever re-inverting from scratch. LSTD(λ) generalises A, b using eligibility-trace-weighted sums in place of single-step terms, exactly paralleling Definition 7.5.2.

Remark. Convergence for these linear least-squares algorithms mirrors Table 10.1-adjacent intuition: LSMC converges on- or off-policy, table-lookup or linear (it is a true least-squares fit, with no bootstrapping); LSTD/LSTD(λ) converge on-policy for table lookup or linear features, but — inheriting the deadly-triad risk of Section 11.6 — are not guaranteed to converge off-policy.

11.7.3 Least Squares Policy Iteration (LSPI)

Extend the action-value analogue directly: features $x(s, a)$, $\hat{q}(s, a, w) = x(s, a)^\top w$, and **LSTDQ** solves the same linear system with $A = \sum_t x(s_t, a_t)(x(s_t, a_t) - \gamma x(s_{t+1}, \pi(s_{t+1})))^\top$, $b = \sum_t x(s_t, a_t) R_{t+1}$, where $\pi(s_{t+1}) = \arg \max_{a'} \hat{q}(s_{t+1}, a', w)$ is whatever policy is currently being evaluated.

Algorithm 7: Least Squares Policy Iteration (LSPI)

- 1 Collect (or reuse) a fixed batch of experience $D = \{(s_t, a_t, r_{t+1}, s_{t+1})\}$;
- 2 Initialise w arbitrarily; $\pi \leftarrow$ any policy;
- 3 **repeat**
 - 4 // Policy evaluation: LSTDQ w.r.t. the current π , re-using D
 - Compute $A = \sum_{(s,a,r,s') \in D} x(s, a)(x(s, a) - \gamma x(s', \pi(s'))^\top$,
 - $b = \sum_{(s,a,r,s') \in D} x(s, a) r$;
 - $w \leftarrow A^{-1}b$;
 - // Policy improvement: greedy w.r.t. the newly fitted $\hat{q}(\cdot, \cdot, w)$
 - $\pi(s) \leftarrow \arg \max_a x(s, a)^\top w \quad \forall s$;
- 7 **until** w converges;

Key Idea 11.7.1: Why LSPI is attractive

LSPI repeatedly re-evaluates the *same* stored experience D under a sequence of different, improving policies π — an extreme form of the experience-replay idea, and (for linear features) the most sample-efficient algorithm in this course: no step-size α to tune at all, since

each evaluation step is an exact least-squares solve rather than a gradient step.

Exam Tip 11.7.1: Big picture for this chapter

Everything here is Chapters 6–10 again, with one change: replace the exact tabular update “ $V(S_t) \leftarrow V(S_t) + \alpha(\text{target} - V(S_t))$ ” by a *gradient* step “ $w \leftarrow w + \alpha(\text{target} - \hat{v}(S_t, w))\nabla_w \hat{v}(S_t, w)$,” which for linear $\hat{v}$ further simplifies to $w \leftarrow w + \alpha(\text{target} - x(S_t)^\top w)x(S_t)$. The one genuinely new phenomenon is the **deadly triad** (Key Idea 11.6.1): know which of the three ingredients (function approximation, bootstrapping, off-policy) each algorithm you’ve studied uses, and you can predict, from Table 10.1-style reasoning alone, whether it is safe.

Appendix A

Summary Tables and Exam Cheat-Sheet

A compressed, single-place reference for last-minute revision. Every entry here is derived and explained in full earlier in these notes; use the cross-references to jump back to the derivation if a formula alone isn't enough.

A.1 Notation Glossary

Symbol	Meaning
$\mathcal{S}, \mathcal{A}, \mathcal{R}$	state space, action space, reward space
S_t, A_t, R_t	state, action, reward at time t
$P_{ss'}^a$	$\mathbb{P}[S_{t+1} = s' \mid S_t = s, A_t = a]$
R_s^a	$\mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$
γ	discount factor, $\gamma \in [0, 1]$
G_t	return, $\sum_{k \geq 0} \gamma^k R_{t+k+1}$
$\pi(a \mid s)$	policy (probability of a given s)
$v_\pi(s), q_\pi(s, a)$	state-/action-value function under π
$v_*(s), q_*(s, a)$	optimal state-/action-value function
π_*	an optimal policy
μ	behaviour policy (off-policy setting)
α	step-size / learning rate
ε	exploration probability (ε -greedy)
λ	eligibility-trace / return-mixing parameter, TD(λ)
$E_t(s)$	eligibility trace of s at time t
δ_t	TD error, $R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ (or the Q -analogue)
w	weight vector for function approximation
$x(s)$ or $x(s, a)$	feature vector
$\hat{v}(s, w), \hat{q}(s, a, w)$	approximate value functions

A.2 The Bellman Equation Family

A.3 Every Update Rule in One Table

Name		Equation
MRP Bellman Eq. (Theorem 3.3.1)		$v(s) = R_s + \gamma \sum_{s'} P_{ss'} v(s')$
Expectation, v_π (Theorem 4.3.3)		$v_\pi(s) = \sum_a \pi(a s) (R_s^a + \gamma \sum_{s'} P_{ss'}^a v_\pi(s'))$
Expectation, q_π (Theorem 4.3.4)		$q_\pi(s, a) = R_s^a + \gamma \sum_{s'} P_{ss'}^a \sum_{a'} \pi(a' s') q_\pi(s', a')$
Optimality, v_* (Theorem 4.4.4)		$v_*(s) = \max_a (R_s^a + \gamma \sum_{s'} P_{ss'}^a v_*(s'))$
Optimality, q_* (Theorem 4.4.5)		$q_*(s, a) = R_s^a + \gamma \sum_{s'} P_{ss'}^a \max_{a'} q_*(s', a')$

A.4 Convergence Conditions, at a Glance

- **Iterative Policy Evaluation / Value Iteration** (Theorems 5.2.2 and 5.4.2): converge unconditionally (any v_0), geometrically at rate γ^k , because the Bellman backup is a γ -contraction in the max-norm.
- **Policy Iteration** (Theorem 5.3.1): converges in finitely many iterations for a finite MDP, since each round either strictly improves v_π somewhere or has already reached π_* .
- **GLIE MC Control / Sarsa** (Theorems 8.3.1 and 9.2.1): need (i) GLIE exploration (infinite exploration + greedy in the limit) and (ii) Robbins–Monro step-sizes, $\sum \alpha_t = \infty$, $\sum \alpha_t^2 < \infty$.
- **Q-Learning** (Theorem 10.3.1): needs every (s, a) visited infinitely often and Robbins–Monro step-sizes, but — unlike Sarsa — *not* GLIE on the behaviour policy (it may keep exploring forever and still converge, since its target is already exactly greedy).
- **Value Function Approximation** (Section 11.6): safe if at most two of {function approximation, bootstrapping, off-policy} are present simultaneously; all three together (the deadly triad) can diverge (Baird’s counterexample).

A.5 “If the Exam Asks You to Derive...”

Algorithm	Update rule
Incremental MC	$V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t))$
TD(0)	$V(S_t) \leftarrow V(S_t) + \alpha(R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$
Forward TD(λ)	$V(S_t) \leftarrow V(S_t) + \alpha(G_t^\lambda - V(S_t)), \quad G_t^\lambda = (1 - \lambda) \sum_n \lambda^{n-1} G_t^{(n)}$
Backward TD(λ)	$V(s) \leftarrow V(s) + \alpha \delta_t E_t(s) \quad \forall s, \quad E_t(s) = \gamma \lambda E_{t-1}(s) + \mathbf{1}(S_t = s)$
MC Control	$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(G_t - Q(S_t, A_t))$, then $\pi \leftarrow \varepsilon$ -greedy(Q)
Sarsa	$Q(S, A) \leftarrow Q(S, A) + \alpha(R + \gamma Q(S', A') - Q(S, A))$
Sarsa(λ)	$Q(s, a) \leftarrow Q(s, a) + \alpha \delta_t E_t(s, a) \quad \forall s, a$
Q-Learning	$Q(S, A) \leftarrow Q(S, A) + \alpha(R + \gamma \max_{a'} Q(S', a') - Q(S, A))$
Off-policy TD(0)	$V(S_t) \leftarrow V(S_t) + \alpha\left(\frac{\pi(A_t S_t)}{\mu(A_t S_t)}(R_{t+1} + \gamma V(S_{t+1})) - V(S_t)\right)$
Linear TD(0) w/ VFA	$w \leftarrow w + \alpha(R_{t+1} + \gamma x(S_{t+1})^\top w - x(S_t)^\top w)x(S_t)$
Linear MC w/ VFA	$w \leftarrow w + \alpha(G_t - x(S_t)^\top w)x(S_t)$
LSTD (direct)	$w = A^{-1}b, \quad A = \sum_t x(s_t)(x(s_t) - \gamma x(s_{t+1}))^\top, \quad b = \sum_t x(s_t)R_{t+1}$

Prompt	Go to
“Derive the Bellman equation for an MRP”	Theorem 3.3.1 (split G_t , tower property)
“Derive/prove convergence of iterative policy evaluation”	Theorem 5.2.2 + Theorem 5.2.1 (contraction mapping)
“Prove the policy improvement theorem”	Theorem 5.3.1 (telescoping expectation)
“Prove value iteration converges”	Theorem 5.4.2 (contraction, via Lemma 5.4.1)
“Derive the incremental mean / MC update”	Proposition 6.2.1 (algebraic split of the sum)
“Explain MC vs. TD bias/variance”	Propositions 7.1.1 and 7.1.2
“Work the AB example”	Example 7.2.1 (MC = sample mean of returns; TD = certainty equivalence)
“Derive/prove forward = backward TD(λ)”	Theorem 7.5.1 (coefficient matching)
“Prove ε -greedy policy improvement”	Theorem 8.2.1
“Derive the Q-learning update from the Bellman optimality equation”	Key Idea 10.3.2, Proposition 10.3.1
“Derive linear TD(0)/LSTD”	Proposition 11.3.1, Proposition 11.7.1
“Explain the deadly triad / Baird’s counterexample”	Key Idea 11.6.1, Example 11.6.1

Key Idea A.5.1: The one-paragraph version of the entire course

Every value function obeys a Bellman equation: value = immediate reward + discounted value of what comes next, averaged either over a policy's action choices, or (for control) maximised over them. Dynamic Programming computes this exactly, using a known model, by iterating the equation as a full-width backup until it reaches its unique (contraction-mapping-guaranteed) fixed point. Once the model is unknown, we replace the full expectation with a *sample*: Monte-Carlo samples an entire episode (unbiased, high variance, no bootstrapping); TD samples one step and bootstraps off the current estimate (biased, low variance); TD(λ) interpolates between them via n -step returns and eligibility traces. For control, GPI alternates evaluating the current policy with improving it greedily (with ε -greedy exploration to keep learning): MC control, Sarsa, and Q-learning are exactly this loop with, respectively, MC, TD(0)-on-policy, and TD(0)-off-policy evaluation — Q-learning's off-policy target $\max_{a'} Q(s', a')$ is what lets it learn about the optimal policy while behaving exploratorily. Finally, when $\mathcal{S}$ is too large to tabulate, every rule above generalises by replacing the table entry $V(s)$ or $Q(s, a)$ with a parametric $\hat{v}(s, w)$ or $\hat{q}(s, a, w)$ and taking a gradient step instead of an exact update — safe in most combinations, but capable of diverging under the simultaneous combination of function approximation, bootstrapping, and off-policy training.